

TESI DI LAUREA IN
INFORMATICA

Generazione automatica di moduli per la cattura delle comunicazioni Command-and-Control dei malware

CANDIDATO
Omar Paviotti

RELATORE
Prof. Marino Miculan

CORRELATORE
Dott. Matteo Paier

CONTATTI DELL'ISTITUTO

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Università degli Studi di Udine

Via delle Scienze, 206

33100 Udine — Italia

+39 0432 558400

<https://www.dmif.uniud.it/>

Al mio cane,
per avermi ascoltato mentre ripassavo le lezioni.

Ringraziamenti

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Sommario

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Indice

1	Introduzione	1
1.1	Problema	1
1.2	Struttura della tesi	2
2	Lo stato dell'arte	3
2.1	Malware e botnet	3
2.1.1	Botnet IoT	4
2.1.2	Architettura	4
2.1.3	Ciclo di vita	4
2.2	Malware analysis	5
2.2.1	Analisi statica	6
	Analisi statica del codice	6
	Limitazioni	7
2.2.2	Analisi dinamica	7
2.3	Large Language Model	8
2.3.1	Prompt engineering	9
2.3.2	Agenti AI	9
2.3.3	Model Context Protocol	10
	Architettura	11
2.4	Reverse engineering assistito	11
2.4.1	LLM nella binary analysis	11
2.4.2	LLM nella malware analysis	13
2.4.3	Interazione Persona-LLM nel SRE	15
3	Il problema	17
3.1	Definizione del problema	17
3.2	Input e output del sistema	18
3.2.1	Input	18
3.2.2	Output	18
3.3	Requisiti del problema	19
3.4	Interfaccia del modulo di output	20
3.5	Diagramma di contesto del sistema	22
4	L'architettura proposta	23
4.1	Architettura del sistema	23
4.2	Componenti del sistema	24
4.2.1	Artefatti	24
4.2.2	Componenti decisionali	26
4.2.3	Integrazione MCP	27
4.2.4	Ambienti di esecuzione	28
4.3	Flusso di esecuzione	28
4.4	Tool calling	30

5	L'implementazione	33
5.1	Ambiente di sviluppo	33
5.2	Orchestrazione / Esecuzione	33
5.2.1	Classifier	33
	Virus Total	33
	YARA Rules	33
5.2.2	SRE Agent	34
	Knowledge Base	34
	Selezione Tools Analisi Statica	34
	Selezione Tools Analisi Dinamica	34
5.2.3	Code Agent	34
5.3	SRE Framework	34
5.3.1	Ghidra	34
5.3.2	MCP Plugin	35
5.3.3	Ghidra MCP Server	35
5.4	Ambiente Sandbox	35
5.4.1	Configurazione	36
5.4.2	Sandbox MCP Server	36
5.5	MCP Client	37
5.5.1	LLM-MCP Client	37
5.6	LLM	37
5.6.1	Server	37
5.6.2	Client	38
	Logging	38
5.7	MessageSchema	39
5.7.1	Message Classification	39
5.7.2	JSON Schema	40
	IP Schema	40
	IP Lookup	40
5.8	Code Templates	41
5.8.1	BotNet Module	41
5.8.2	MessageParser Module	42
5.9	Prompt Archive	43
5.9.1	Renaming	43
5.9.2	Reversing	43
5.9.3	Code Agent	43
6	Validazione e test	45
6.1	Obiettivi della validazione	45
6.1.1	Strategia di validazione	45
6.2	Ambiente sperimentale	45
6.2.1	LLM	45
6.3	Definizione delle metriche di valutazione	46
6.3.1	Complessità del campione	46
6.3.2	Efficacia dell'attività di rinomina	46
6.3.3	Accuratezza del recupero degli artefatti	46
6.3.4	Qualità del codice generato	46
6.4	Campagna sperimentale	46
6.4.1	Dataset	46
	Famiglia Mirai	46
	Snippet MD K.B.	46
6.4.2	Descrizione dei campioni	46
	Hash, architettura, dimensione, famiglia	46

	Difficoltà	46
6.5	Risultati sperimentali	46
6.5.1	Statistiche di esecuzione	46
	Token	46
6.5.2	Metriche	46
	Tabelle	46
	Grafici	46
7	Conclusioni	49
7.1	Sviluppi Futuri	49
A	Titolo della prima appendice	51

Elenco delle tabelle

Elenco delle figure

2.1	Ciclo di vita di una botnet	5
2.2	Architettura del Model Context Protocol (MCP)	11
2.3	Approcci LLM4Decompile-End e -Ref per la decompilazione	12
2.4	Panoramica dell'architettura di MASDroid	14
3.1	Interfaccia astratta BotNetInterface	21
3.2	Esempio di uso di un generico modulo BotNet che implementa BotNetInterface	21
3.3	Diagramma di contesto del sistema	22
4.1	Diagramma UML dei componenti dell'architettura proposta	23
4.2	Diagramma UML delle attività della pipeline di analisi	29
4.3	Diagramma UML di sequenza delle operazioni di tool calling	30

1

Introduzione

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

1.1 Problema

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim

sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

1.2 Struttura della tesi

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

2

Lo stato dell'arte

Questo capitolo ha l'obiettivo di presentare i principali concetti teorici e le nozioni fondamentali necessarie per comprendere gli argomenti trattati nei capitoli successivi. Saranno quindi introdotti gli aspetti essenziali su cui si fonda lo sviluppo del lavoro descritto in questa tesi.

2.1 Malware e botnet

In generale, qualsiasi software progettato per compromettere la riservatezza, l'integrità o la disponibilità di un sistema informatico, di una rete o dei dati di un utente può essere classificato come *malware*, abbreviazione di *malicious software*. Il malware può assumere diverse forme e può essere sviluppato con differenti finalità, tra cui il furto di informazioni, il danneggiamento dei sistemi o l'accesso a dispositivi non autorizzato. In base al proprio obiettivo, un malware può appartenere a una o più categorie, tra cui, ad esempio, *backdoor*, *information stealer*, *launcher*, *worm*, *virus* e altre [1].

Nel corso degli anni, l'evoluzione delle infrastrutture di rete e il cambiamento delle abitudini tecnologiche hanno portato a un aumento del numero di dispositivi connessi e del tempo durante il quale essi rimangono online. Questo fenomeno, unito alla presenza di vulnerabilità di sicurezza nei sistemi, ha favorito lo sviluppo e la diffusione di nuove minacce informatiche, tra cui le *botnet*, oggi considerate una delle principali minacce nel panorama della sicurezza informatica.

Una botnet è una rete di host compromessi, detti *bot*, controllati remotamente da uno o più attaccanti, chiamati *botmaster*. Il malware installato sul dispositivo consente al botmaster di controllare da remoto i dispositivi e di eseguire azioni malevole. [2]

Dal punto di vista degli attaccanti, le botnet rappresentano uno strumento estremamente redditizio, poiché consentono di ottenere un ritorno economico attraverso lo sfruttamento dei dispositivi compromessi. Una volta infettati, tali dispositivi possono essere impiegati simultaneamente per eseguire attività malevole su larga scala, generando danni economici per utenti, aziende e Internet Service Provider (ISP) e fungendo da infrastruttura per numerose attività criminali online.

Le botnet possono essere impiegate per numerose attività, tra cui l'invio di spam, la distribuzione di malware, il furto di informazioni, frodi informatiche, estorsioni e attacchi *Distributed Denial of Service* (DDoS). Tra queste, gli attacchi DDoS rappresentano uno degli utilizzi più diffusi, poiché consentono di

sfruttare la capacità trasmissiva complessiva dei dispositivi compromessi per sovraccaricare il bersaglio e rendere i suoi servizi indisponibili.

2.1.1 Botnet IoT

Parallelamente alla crescita delle botnet, si sono evolute anche le tecniche di difesa, che possono essere di tipo reattivo, intervenendo dopo la compromissione, oppure preventivo, con l'obiettivo di impedire o limitare l'infezione prima che questa si verifichi. Nonostante il progressivo rafforzamento dei meccanismi di protezione, la continua evoluzione delle tecnologie e dei dispositivi connessi ha offerto nuove opportunità agli attaccanti, contribuendo all'espansione della superficie d'attacco. In particolare, la rapida diffusione dei dispositivi appartenenti all'*Internet of Things* (IoT) ha favorito la creazione e la diffusione di botnet su larga scala, a causa delle risorse hardware limitate e di configurazioni di sicurezza spesso inadeguate.

Nel 2016 Internet è stata interessata dalla diffusione della botnet *Mirai*, composta principalmente da dispositivi embedded e IoT compromessi, che nel giro di poco tempo è stata responsabile di alcuni dei più grandi attacchi DDoS mai registrati.

La rapida espansione di *Mirai* è stata resa possibile dalla combinazione di diversi fattori, tra cui un'implementazione relativamente semplice del malware, una scansione estremamente efficiente della rete, la capacità di infettare dispositivi eterogenei e, soprattutto, lo sfruttamento della presenza di credenziali predefinite deboli o mai modificate sui dispositivi IoT. [3]

2.1.2 Architettura

Il componente fondamentale di una botnet è l'infrastruttura di *Command-and-Control* (C&C), che permette la comunicazione tra il botmaster e i bot compromessi, coordinandone le attività. Attraverso il canale C&C vengono trasmessi i comandi, che i bot interpretano ed eseguono sul dispositivo infetto.

Per realizzare tale comunicazione possono essere impiegati diversi protocolli di rete. Oltre alla trasmissione dei comandi, il canale C&C consente di mantenere il controllo dei bot, distribuire aggiornamenti del malware e raccogliere informazioni dai dispositivi compromessi. I protocolli utilizzati, così come le funzionalità implementate, variano in funzione degli obiettivi della botnet.

L'architettura dell'infrastruttura C&C influenza direttamente la robustezza e la capacità di sopravvivenza della botnet. Le principali architetture adottate sono quelle centralizzate, caratterizzate da un modello *client-server* con uno o pochi server C&C, e quelle decentralizzate, generalmente basate su reti *peer-to-peer* (P2P), ma esistono anche architetture ibride che combinano caratteristiche di entrambi gli approcci [2].

2.1.3 Ciclo di vita

Il ciclo di vita di una botnet può essere generalmente suddiviso in cinque fasi concatenate, secondo il modello proposto in [2]:

Infezione iniziale. Un host viene compromesso attraverso dei vettori di attacco comuni e diventa un potenziale bot.

Iniezione secondaria. Il malware iniziale, che può essere un *dropper* o un *downloader*, scarica o attiva il componente principale della botnet, trasformando il dispositivo in un bot controllabile dal botmaster.

Connessione C&C. Il nuovo bot deve stabilire una connessione comunicazione con l'infrastruttura C&C per registrare la propria presenza e per mettersi in attesa di istruzioni o aggiornamenti. La risoluzione dell'indirizzo del server C&C avviene tramite metodi statici (indirizzi IP o nomi di dominio *hardcoded* nel binario) oppure mediante tecniche più sofisticate, come i *Domain Generation Algorithm* (DGA).

Attività malevole. Il bot è ora nella fase operativa, è pronto per ricevere i *payload* di comando ed eseguire le attività malevole ordinate dal botmaster.

Manutenzione e aggiornamento. Alla fine la botnet entra nella fase di manutenzione, che ha l'obiettivo di garantire la sopravvivenza della botnet nel lungo periodo. Il botmaster può inviare aggiornamenti del malware, con lo scopo di correggere vulnerabilità o aggiungere nuove funzionalità.

Il modello C&C introduce però una vulnerabilità intrinseca: la necessità per i bot di comunicare con l'infrastruttura di comando espone l'attività malevola all'osservazione, costituendo un punto debole sfruttato dalle metodologie di rilevamento. Il traffico analizzato include l'*handshake* iniziale (scambio di identificativi ed eventuale *profiling*, ovvero la raccolta di informazioni sulle caratteristiche dell'host compromesso), il *beaconing* periodico verso un *endpoint* C&C (che funge da *keepalive* per segnalare la disponibilità del bot) e i *payload* di comando.

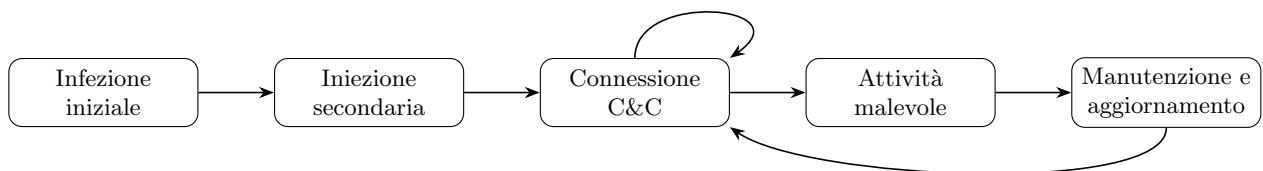


Figura 2.1: Ciclo di vita di una botnet [2]

2.2 Malware analysis

La *malware analysis* (analisi del malware) è la disciplina che si occupa di analizzare i malware al fine di identificarne le funzionalità, comprenderne il comportamento e sviluppare adeguate contromisure per il rilevamento e la mitigazione delle minacce [1].

Nel contesto della risposta a un incidente di sicurezza, la malware analysis consente di raccogliere le informazioni necessarie per ricostruire quanto accaduto, determinando gli obiettivi, le capacità e il comportamento del malware. Tra i risultati dell'analisi vi è anche l'individuazione di elementi utili al rilevamento, tra cui le *signatures* e gli *Indicators of Compromise* (IoC). Questi ultimi possono essere di tipo *host-based*, quando riguardano artefatti presenti sul sistema compromesso, come file creati o

modificati e variazioni al registro di sistema, oppure *network-based*, quando sono ricavati dall'analisi del traffico di rete e consentono di individuare comunicazioni riconducibili all'attività del malware.

Nella maggior parte dei casi, chi conduce l'analisi dispone esclusivamente del file eseguibile del malware, mentre il codice sorgente non è disponibile. Di conseguenza, comprenderne il funzionamento richiede l'impiego di tecniche di *Software Reverse Engineering* (SRE), che consentono di ricostruire la logica del programma a partire dal codice compilato. Le tecniche di SRE si dividono principalmente in due categorie, analisi statica e analisi dinamica, basate su approcci complementari tra loro.

2.2.1 Analisi statica

La *static analysis* consiste nell'esaminare un malware senza eseguirlo, analizzandone la struttura e il contenuto al fine di ricavarne informazioni sul funzionamento.

Le tecniche di analisi statica di base comprendono l'utilizzo di strumenti antivirus per una prima classificazione del campione, il calcolo di hash per identificarne univocamente il file e l'ispezione delle informazioni contenute nell'eseguibile, quali stringhe, intestazioni del formato binario, librerie importate o esportate e altri metadati utili. Sebbene queste tecniche consentano di ottenere rapidamente informazioni preliminari, da sole raramente permettono di comprendere completamente il comportamento del malware [1].

Analisi statica del codice

L'analisi statica avanzata si basa sull'ispezione delle istruzioni contenute nel file eseguibile del malware. Poiché tali istruzioni sono espresse in linguaggio macchina e non sono direttamente leggibili dal reverse engineer, viene utilizzato un *disassembler*, uno strumento che traduce il codice macchina nel corrispondente linguaggio assembly. Quest'ultimo fornisce una rappresentazione testuale delle istruzioni della CPU, permettendo di analizzare le operazioni eseguite dal programma a basso livello [1].

Un ulteriore strumento utilizzato nell'analisi dei binari è il *decompiler*, che tenta di tradurre il codice macchina in una rappresentazione equivalente in un linguaggio di alto livello. Sebbene il risultato non corrisponda necessariamente al codice sorgente originale, questa rappresentazione introduce un maggiore livello di astrazione e può facilitare la comprensione della logica del programma.

Due dei più noti strumenti di RE sono IDA Pro e Ghidra. Questi software offrono un ambiente completo per l'analisi dei binari, affiancando al disassembler funzionalità di decompilazione, *debugging* e strumenti di supporto all'analisi statica come la navigazione del codice, l'analisi delle funzioni e la ricostruzione dei tipi. Tuttavia, le esigenze degli sviluppatori e dei reverse engineer possono richiedere l'introduzione di funzionalità aggiuntive non direttamente disponibili. Per questo motivo Ghidra, così come altri strumenti di SRE, mette a disposizione meccanismi di scripting basati sull'*Application Programming Interface* (API) della piattaforma, che consente di interagire con le funzionalità interne del software. Attraverso tale API è possibile creare ed eseguire script in Java e Python. Questi possono variare da semplici automazioni di operazioni comuni fino alla realizzazione di strumenti più complessi per l'analisi dei programmi, arrivando anche a estendere le capacità della piattaforma mediante plugin dedicati [4].

Limitazioni

L'analisi statica del codice consente di ottenere una comprensione molto più approfondita del malware rispetto alle tecniche di analisi di base. Tuttavia, presenta alcune limitazioni significative.

Una prima limitazione è rappresentata dall'elevato livello di competenza richiesto per svolgere attività di SRE. Il reverse engineer deve possedere una solida conoscenza dell'architettura del processore, del linguaggio assembly, delle convenzioni di chiamata e dei meccanismi con cui i compilatori traducono i costrutti ad alto livello in codice macchina, oltre a una buona esperienza nell'utilizzo degli strumenti di analisi.

Una seconda limitazione è intrinseca all'analisi statica stessa. Alcuni comportamenti del malware dipendono infatti da informazioni disponibili esclusivamente a runtime, come parametri di esecuzione, configurazione del sistema, dati ricevuti dalla rete o indirizzi risolti dinamicamente, che non possono essere determinati osservando unicamente il codice.

A tali limitazioni si aggiungono numerose tecniche impiegate dagli autori dei malware con l'obiettivo di ostacolare le attività di SRE. Tra le più comuni rientra la *code obfuscation*, che modifica la rappresentazione e la struttura del programma preservandone al contempo il comportamento. L'offuscamento può riguardare il flusso di controllo (*control-flow obfuscation*), attraverso trasformazioni che rendono più complessa la ricostruzione delle relazioni tra le istruzioni; la posizione dei dati (*data-location obfuscation*), rendendo più difficile determinare dove sono memorizzate variabili e altre informazioni utilizzate dal programma; oppure l'utilizzo dei dati (*data-usage obfuscation*), ostacolando la ricostruzione delle relazioni tra la definizione e il successivo utilizzo dei valori [5].

Un'ulteriore tecnica è quella del *self-modifying code*, nella quale il malware genera, decrittata o modifica porzioni del proprio codice durante l'esecuzione, rendendo impossibile osservarne staticamente la versione definitiva. Molto diffuso è il *packing*, che trasforma il programma originale mediante compressione, cifratura o altre tecniche, sostituendolo con un eseguibile contenente solamente uno *stub* incaricato di ripristinare il codice originale durante l'esecuzione. Di conseguenza, prima di poter analizzare il malware è necessario recuperarne il codice originale mediante una fase preliminare di *unpacking* [6].

Infine, molti malware vengono distribuiti solo dopo essere stati sottoposti a un processo di *stripping*, attraverso il quale vengono rimossi simboli, nomi delle funzioni, informazioni di debug e altri metadati dal file eseguibile. La conseguente perdita di informazioni semantiche rende più difficile la ricostruzione della logica del programma durante il processo di SRE.

Poiché tali tecniche sono ampiamente note e consolidate, è comune che i malware moderni ne adottino una o più contemporaneamente, con l'obiettivo di aumentare la complessità e il tempo richiesti per l'analisi del campione.

2.2.2 Analisi dinamica

La *dynamic analysis* consiste nell'analizzare il malware durante la sua esecuzione, osservandone direttamente il comportamento.

L'esecuzione deve ovviamente avvenire all'interno di un ambiente controllato e progettato appositamente per l'analisi, in modo da evitare che il malware possa compromettere il sistema del reverse engineer o la rete circostante. Nella pratica vengono impiegate prevalentemente macchine virtuali, mentre l'uti-

lizzo di sistemi fisici isolati (*air-gapped*) è meno frequente. L'ambiente di esecuzione viene generalmente configurato in funzione degli obiettivi dell'analisi e, in alcuni casi, come nello studio delle botnet, è necessario abilitare l'accesso a Internet. Questa scelta espone il reverse engineer ai rischi connessi all'interazione del malware con l'esterno, come la possibilità di allertare gli autori riguardo all'attività di analisi o, nel caso peggiore, di consentire inconsapevolmente l'esecuzione delle attività malevole per cui il malware è stato progettato.

L'esecuzione del malware all'interno di una *sandbox*, ovvero un ambiente protetto, consente di monitorare numerosi aspetti del suo comportamento, tra cui i processi creati, le operazioni sul file system, le modifiche al registro di sistema, le connessioni di rete e il traffico generato, che può essere catturato e analizzato mediante strumenti di *packet sniffing* come Wireshark.

Un livello di analisi ancora più approfondito è rappresentato dal debugging. Un *debugger* è uno strumento software utilizzato per osservare e controllare l'esecuzione di un programma, fornendo piena visibilità sul suo stato interno durante l'esecuzione. Esso permette ai reverse engineer di ispezionare il contenuto delle locazioni di memoria, i registri del processore e gli argomenti delle funzioni, oltre a seguire il flusso di esecuzione del programma. Inoltre, i debugger consentono di modificare dinamicamente lo stato del programma, permettendo di analizzarne specifici comportamenti o di verificarne il funzionamento in particolari condizioni [1].

Rispetto alla sola analisi statica, il debugging permette quindi di osservare informazioni disponibili esclusivamente a runtime. Tuttavia, anche questo approccio presenta alcune limitazioni. Oltre ai vincoli imposti dall'ambiente di esecuzione, molti malware implementano tecniche di *anti-debugging* e *anti-virtualization*, mediante le quali rilevano di essere eseguiti rispettivamente sotto il controllo di un debugger o all'interno di una macchina virtuale. In tali casi, il malware può modificare il proprio comportamento o interrompere completamente l'esecuzione, con l'obiettivo di ostacolare l'analisi.

2.3 Large Language Model

I *Large Language Model* (LLM) appartengono alla famiglia della *Generative Artificial Intelligence* (GenAI), un sottocampo dell'intelligenza artificiale che comprende modelli in grado di generare nuovi contenuti, quali testo, immagini, audio e dati sintetici, a partire dai pattern appresi durante l'addestramento su grandi quantità di dati. I LLM rappresentano la specializzazione della GenAI nel dominio del linguaggio naturale.

Un LLM è un modello di *Deep Learning*, basato su reti neurali artificiali di grandi dimensioni e addestrato su vaste collezioni di testo mediante tecniche di apprendimento auto-supervisionato e semi-supervisionato. Durante il *training*, il modello apprende le strutture e i pattern linguistici, acquisendo la capacità di comprendere e generare testo in linguaggio naturale.

L'architettura alla base dei moderni LLM è il *Transformer*, introdotto da Google nel 2017. Il Transformer si basa sul meccanismo di *self-attention*, che consente al modello di individuare le relazioni tra i diversi *token*, le unità in cui viene suddiviso il testo. A ciascun token viene associata una rappresentazione numerica, detta *embedding*, che ne cattura le caratteristiche semantiche e linguistiche. Questi modelli adottano prevalentemente un'architettura *decoder-only*, in cui il testo viene generato in modo autoregressivo, prevedendo un token alla volta sulla base di quelli precedentemente elaborati [7].

Questa architettura, combinata con la natura *general-purpose* dei LLM li rende utilizzabili per un ampio insieme di compiti. Tra le attività più comuni rientrano la conversazione in linguaggio naturale, la risposta a domande, la classificazione del testo, la traduzione automatica e il riassunto. I LLM vengono inoltre impiegati per attività più complesse, come la generazione di codice, la produzione di contenuti testuali e l'esecuzione di compiti di ragionamento, basati sulla capacità di trarre conclusioni a partire dalle informazioni fornite.

Nonostante i notevoli progressi degli ultimi anni, i LLM presentano ancora importanti limitazioni. Tra le principali vi sono la disponibilità e la qualità dei dati di addestramento, con problematiche legate alla privacy, alla presenza di contenuti protetti da copyright e alla rappresentatività dei dati. Oltre a questi aspetti, rimangono criticità legate all'introduzione di *bias* derivanti dai dati utilizzati durante il training e alla possibilità di produrre informazioni errate o inventate, con la generazione di risposte plausibili ma fattualmente scorrette, fenomeno noto come *hallucination* (allucinazione) [7].

2.3.1 Prompt engineering

Inizialmente, l'adattamento di un LLM a compiti specifici richiedeva generalmente il *fine-tuning* del modello, ovvero un ulteriore addestramento su dati appositamente selezionati. Oggi, il *prompt engineering* consente di orientarne il comportamento attraverso la composizione di *prompt*, ossia gli input forniti al modello. Anche piccole variazioni nella loro formulazione possono produrre differenze significative nella qualità e nella tipologia delle risposte generate.

Il *prompt engineering* è la pratica di progettare e formulare dei prompt che consentano ai LLM di raggiungere specifici obiettivi. L'obiettivo del prompt engineering è quello di tradurre l'intento dell'utente in istruzioni chiare e strutturate, che il modello possa interpretare correttamente durante la fase di inferenza, cioè la fase in cui il modello genera la risposta.

Oltre alle istruzioni esplicite, assume un ruolo importante anche il contesto fornito al modello e la cronologia della conversazione. Le principali tecniche di prompting sono [7]:

Zero-shot prompting. Il modello riceve esclusivamente le istruzioni necessarie per svolgere il compito, senza alcun esempio. La risposta si baserà interamente sulla conoscenza acquisita dal LLM durante l'addestramento.

Few-shot prompting. Il modello, oltre al compito da svolgere, riceve un piccolo gruppo di esempi, con l'obiettivo di mostrargli il formato o il pattern da seguire durante la generazione della risposta.

Multi-turn prompting. Il modello riceve anche la cronologia dei turni conversazionali precedenti, mantenendo il contesto dell'interazione, così da avere conversazioni più naturali e coerenti. Tuttavia, conversazioni molto lunghe possono introdurre difficoltà legate ai limiti della finestra di contesto e alla riduzione dell'attenzione del modello verso parte dell'informazione.

2.3.2 Agenti AI

L'evoluzione dei LLM è stata inizialmente guidata principalmente dallo *scaling* dei modelli, aumentando il numero di parametri, la quantità di dati di addestramento e le risorse computazionali impiegate durante il training. Più recentemente, la ricerca si è concentrata anche sul miglioramento delle capacità

di ragionamento durante la fase di inferenza, mostrando come dedicare più tempo al processo di *reasoning* (ragionamento) prima della generazione della risposta possa migliorare le prestazioni del modello nei compiti più complessi.

Questo approccio ha favorito lo sviluppo di modelli addestrati a produrre esplicitamente catene di ragionamento (*Chain of Thought*, CoT), ossia sequenze di passaggi logici che guidano il modello nella risoluzione di un problema prima di formulare la risposta finale. Sebbene tutti i più recenti LLM dispongano di avanzate capacità di reasoning, affidarsi esclusivamente al ragionamento interno del modello presenta un'importante limitazione, poiché il modello può costruire catene logiche coerenti senza poter necessariamente verificarne la correttezza attraverso fonti di informazione o strumenti esterni, aumentando così il rischio di produrre allucinazioni. Allo stesso tempo, consentire a un modello di interagire direttamente con un ambiente esterno (*acting*) senza un adeguato processo di ragionamento può portare a decisioni meno accurate, errori di pianificazione e un utilizzo inefficace degli strumenti disponibili.

Al fine di superare queste limitazioni complementari è stato proposto il framework *ReAct* (*Reasoning and Acting*), con l'obiettivo di risolvere compiti complessi combinando il ragionamento del modello con l'esecuzione di azioni specifiche per il compito da svolgere [8]. Il funzionamento si basa su un ciclo iterativo composto da tre fasi:

Thought. (Ragionamento) Il modello genera un ragionamento testuale sullo stato corrente del problema, analizzando le informazioni disponibili e pianificando l'azione successiva.

Action. (Azione) Il modello produce un'azione strutturata, tipicamente sotto forma di invocazione di uno strumento (tool) o di una funzione disponibile nell'ambiente esterno.

Observation. (Osservazione) L'ambiente esterno esegue l'azione richiesta e restituisce il risultato al modello, che viene quindi fornito come input per l'iterazione successiva.

Il paradigma ReAct costituisce la base concettuale della maggior parte dei moderni agenti AI. Un agente AI è un agente intelligente progettato per perseguire uno o più obiettivi attraverso un ciclo continuo di ragionamento e azione, con diversi livelli di autonomia, interagendo con strumenti e ambienti esterni. In questo contesto, il LLM costituisce il motore decisionale dell'agente, che può essere integrato con componenti aggiuntivi, quali memoria persistente, logiche di pianificazione e un software di orchestrazione, detto *agent harness*, responsabile del coordinamento delle diverse componenti e della loro esecuzione.

2.3.3 Model Context Protocol

Il *Model Context Protocol* (MCP) è uno standard *open-source*, introdotto da Anthropic nel 2024, che permette di connettere le *AI application* con sistemi esterni [9]

L'obiettivo del MCP è quello di fornire un'interfaccia standardizzata per collegare le AI application a fonti di informazioni, strumenti e altri flussi di lavoro, consentendo ai LLM di accedere a dati esterni e di svolgere operazioni attraverso sistemi terzi.

Il MCP è stato introdotto per risolvere il problema dell'integrazione tra AI application e sistemi esterni. Prima della sua introduzione, gli sviluppatori dovevano implementare connettori specifici per

ciascuna sorgente di dati o strumento, spesso dipendenti dal singolo fornitore (*vendor-specific*), con conseguenti problemi di integrazione e interoperabilità.

Architettura

Il protocollo distingue tre ruoli principali:

MCP Host. L'AI application, come un ambiente di sviluppo o un chatbot avanzato in cui risiede il LLM e che coordina il flusso di lavoro, creando uno o più Client per interfacciarsi con i Server.

MCP Client. Un modulo integrato nell'Host che agisce come intermediario, mantenendo una connessione uno-a-uno con un singolo Server, instradando le richieste dell'Host e restituendo i risultati ottenuti.

MCP Server. Un programma che espone funzionalità specifiche a uno o più Client. Ogni server può mettere a disposizione una o più *capability* (capacità), classificate in *tools* e *resources*.

I *tools* rappresentano operazioni invocabili dal LLM e sono descritti tramite interfacce basate su JSON Schema, che definiscono in modo esplicito gli input e gli output previsti. Le *resources*, invece, forniscono accesso strutturato a informazioni esterne; sono identificate mediante URI, associate a un tipo MIME e possono essere fornite al LLM come contesto aggiuntivo durante l'esecuzione.

Dal punto di vista dei livelli di comunicazione, il *data layer* definisce il protocollo di scambio tra Client e Server, basato su JSON-RPC 2.0, specificando il formato dei messaggi e la relativa semantica. Il *transport layer*, invece, definisce il meccanismo di trasporto e il canale attraverso cui avviene lo scambio dei messaggi, occupandosi della gestione della connessione e del *message framing*. Attualmente il protocollo supporta due meccanismi di trasporto: lo *stdio transport*, che utilizza gli stream di input e output standard per la comunicazione tra processi locali sulla stessa macchina e lo *Streamable HTTP transport*, basato su richieste HTTP, che consente la comunicazione con server remoti.

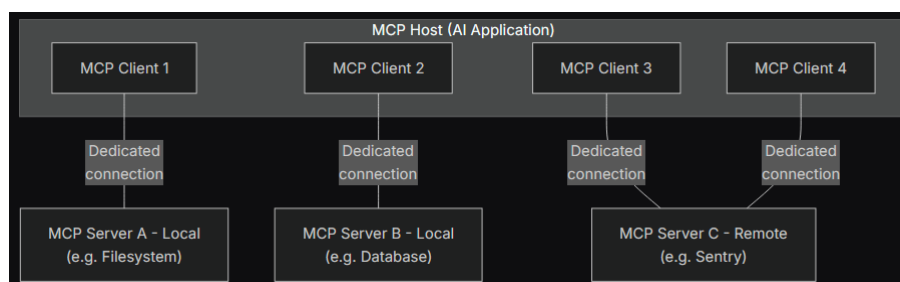


Figura 2.2: Architettura del Model Context Protocol (MCP) [9]

2.4 Reverse engineering assistito

2.4.1 LLM nella binary analysis

I rapidi progressi nel campo dell'AI hanno portato all'impiego degli LLM anche nel SRE, dove le loro capacità di comprensione e generazione del codice possono essere sfruttate dai reverse engineer per supportare il processo di analisi binaria dei software.

In questo contesto si inseriscono i due lavori descritti di seguito.

LLM4Decompile [10]. Propone una famiglia di LLM open-source, con dimensioni comprese tra 1.3 e 33 miliardi di parametri, *fine-tuned* per la decompilazione di codice binario in linguaggio C. L'obiettivo è produrre codice sorgente più leggibile e semanticamente corretto rispetto ai decompilatori tradizionali.

Lo studio introduce due classi di modelli, che seguono approcci complementari:

- *LLM4Decompile-End*. Dopo il disassemblaggio del codice binario in istruzioni assembly x86, il modello genera direttamente il corrispondente codice C, ottenendo risultati superiori sia al decompilatore di Ghidra sia a modelli general-purpose come GPT-4o.
- *LLM4Decompile-Ref*. In questo caso, dopo il disassemblaggio, il codice viene inizialmente decompilato mediante Ghidra; successivamente, il modello utilizza il codice decompilato come punto di partenza per migliorarne leggibilità e correttezza semantica. L'intervento non si limita alla semplice revisione dei nomi e dei tipi delle variabili, ma mira a una rielaborazione più generale del codice, aumentando la potenziale eseguibilità rispetto alla sola decompilazione automatica.

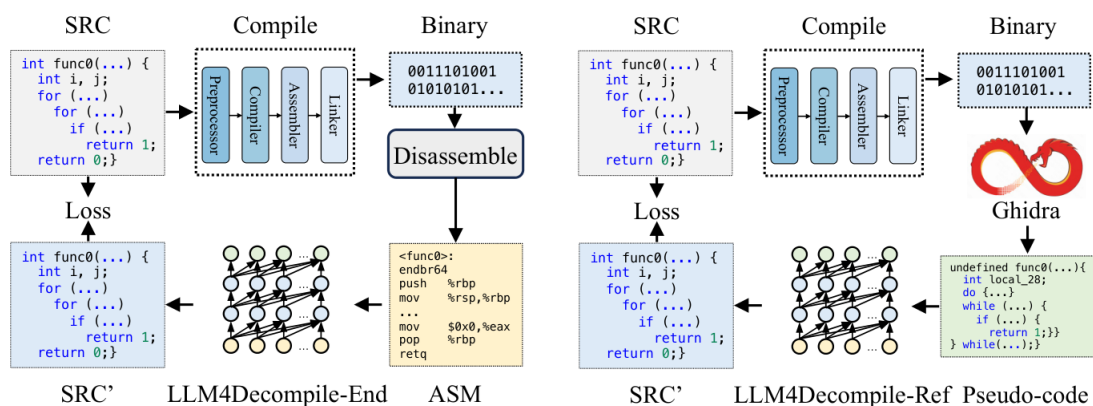


Figura 2.3: Approcci LLM4Decompile-End e -Ref per la decompilazione [10]

I risultati sperimentali ottenuti con LLM4Decompile mostrano come LLM specializzati possano superare gli strumenti tradizionali di decompilazione, evidenziando il potenziale degli LLM come supporto alle attività di SRE.

ReCopilot [11]. Presenta un LLM specializzato nelle principali attività dell'analisi statica, progettato per operare come un copilota a supporto del reverse engineer durante il processo di analisi.

ReCopilot adotta un approccio di addestramento articolato in più fasi:

- *Continue Pre-Training* (CPT). Un LLM di base viene ulteriormente pre-addestrato su un ampio corpus di dati relativi al SRE, al fine di acquisire una conoscenza specialistica del dominio e delle relazioni tra codice binario, assembly, codice sorgente e linguaggio naturale.
- *Supervised Fine-Tuning* (SFT). Il modello viene quindi specializzato su un insieme di task tipiche dell'analisi statica, imparando a seguire le istruzioni dell'utente e a produrre output

in formati strutturati. Tra le attività considerate rientrano la decompilazione, l'analisi delle funzioni e dei relativi nomi, parametri e variabili, a cui si aggiungono task come l'identificazione di algoritmi, la classificazione e la generazione di riassunti relativi a codice e funzioni. Il training include inoltre esempi basati su Chain of Thought, con l'obiettivo di guidare il modello nel processo di ragionamento prima della generazione della risposta.

- *Direct Preference Optimization* (DPO). Il modello viene infine ottimizzato rispetto alle preferenze umane, migliorandone la capacità di seguire il formato richiesto e la qualità, consistenza e pertinenza delle risposte generate.

Dal punto di vista sperimentale, ReCopilot supera sia gli strumenti tradizionali di SRE, come IDA Pro e Ghidra, sia gli LLM general-purpose, pur mantenendo dimensioni contenute (7 miliardi di parametri rispetto alle dimensioni molto maggiori dei modelli general-purpose). Inoltre, la natura specialistica e strutturata del modello lo rende particolarmente adatto all'integrazione all'interno di sistemi agentici. In questo contesto, ReCopilot può essere impiegato come modulo dedicato all'analisi binaria da parte di un agente di SRE autonomo, al quale può fornire analisi dettagliate a partire da pseudocodice o istruzioni assembly.

2.4.2 LLM nella malware analysis

Dopo le prime applicazioni degli LLM al SRE in contesti generici, il loro impiego è stato progressivamente esteso anche ad ambiti più specifici della cybersecurity, tra cui la malware analysis.

A questo scopo, sono emerse soluzioni specializzate, progettate per sfruttare le capacità degli LLM a supporto dell'analisi dei campioni malevoli e delle attività di SRE, come le due soluzioni presentate di seguito.

Project IRE [12]. Sviluppato da Microsoft, rappresenta uno dei primi esempi industriali di applicazione degli LLM alla malware analysis. Il sistema è progettato per classificare automaticamente un file eseguibile come benigno o maligno disponendo esclusivamente del campione binario, senza fare affidamento su metadati, telemetria o altre informazioni relative alla sua provenienza o al suo scopo. L'obiettivo è assistere gli analisti nella gestione dell'elevato numero di file da analizzare, riducendo i tempi e il carico operativo.

Project IRE adotta un'architettura agentic nella quale un LLM coordina una suite di strumenti specializzati per il SRE. L'architettura consente di combinare diversi livelli di analisi, passando dall'analisi binaria a basso livello alla ricostruzione del control flow, fino all'interpretazione ad alto livello del comportamento del codice. I diversi strumenti, accessibili tramite API, includono sandbox per l'analisi dei campioni, strumenti Microsoft e componenti open source, tra cui decompilatori, strumenti per la ricerca e per il recupero di documentazione.

Durante l'analisi, il sistema costruisce una *chain of evidence*, ovvero una sequenza di evidenze che documenta il percorso seguito per giungere alla classificazione finale. Questa traccia risulta particolarmente utile durante la revisione da parte degli analisti, poiché consente di verificare le evidenze alla base della decisione e di individuare i passaggi che possono aver contribuito a un'eventuale classificazione errata.

Le valutazioni preliminari riportano un tasso di classificazione corretta pari a circa il 90% dei campioni analizzati, con una percentuale di falsi positivi prossima al 2%. Questi risultati suggeriscono la possibilità di impiegare il sistema in scenari operativi, mantenendo tuttavia la revisione da parte di analisti esperti come componente del processo di analisi.

MASDroid [13]. Propone un framework multi-agente dedicato all'analisi di malware Android. Il sistema combina diversi agenti specializzati con tecniche di analisi comportamentale, producendo una classificazione del malware accompagnata da spiegazioni interpretabili dal reverse engineer.

L'obiettivo del framework è superare alcune delle principali limitazioni degli approcci esistenti, tra cui la possibilità di elusione delle tecniche di analisi puramente statica, il costo computazionale maggiore dell'analisi dinamica e, soprattutto, la limitata trasparenza dei processi decisionali dei sistemi tradizionali.

L'architettura di MASDroid può essere suddivisa in tre componenti principali:

- *Static Analysis*: il file APK viene preliminarmente decompilato e analizzato, estraendo un insieme di caratteristiche che viene successivamente fornito ai diversi agenti come base comune per l'analisi.
- *Multi-Agent Collaboration*: i diversi agenti operano in parallelo, ciascuno responsabile di una specifica attività di analisi, per cui può utilizzare i tool a disposizione. Gli output prodotti vengono organizzati in un formato strutturato, che combina un *maliciousness score* con una *evidence list*, e vengono successivamente forniti all'agente responsabile della decisione finale.
- *Decision Agent*: l'agente di decisione integra le informazioni provenienti dai diversi agenti ed esegue la classificazione finale. Esso genera inoltre un report strutturato contenente lo score finale, i riferimenti alle evidenze e alle attività dei singoli agenti, rendendo il processo decisionale maggiormente interpretabile dal reverse engineer.

Le valutazioni sperimentali mostrano prestazioni superiori al 97% sia nella rilevazione del malware sia nella classificazione della relativa famiglia. Oltre ai risultati quantitativi, gli autori evidenziano come l'approccio multi-agente migliori l'interpretabilità delle decisioni rispetto ai tradizionali modelli *black-box*, fornendo classificazioni supportate dalle evidenze raccolte durante il processo di analisi.

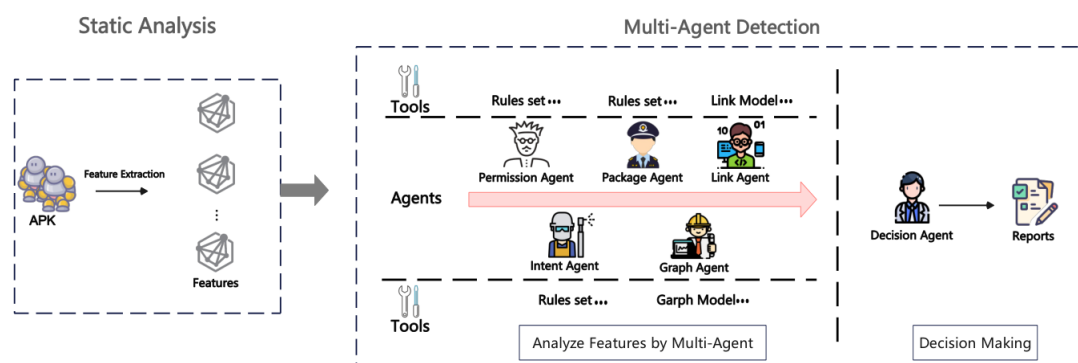


Figura 2.4: Panoramica dell'architettura di MASDroid [13]

2.4.3 Interazione Persona-LLM nel SRE

La crescente diffusione degli LLM nelle attività di SRE evidenzia come lo stato dell'arte della ricerca si stia progressivamente orientando verso l'integrazione di tali modelli all'interno del flusso di lavoro dei reverse engineer.

In questo contesto che, tra il 2024 e il 2025, Basque et al. [14] hanno condotto il primo studio sistematico volto a comprendere come gli LLM possano collaborare con i reverse engineer durante le attività di SRE. Lo studio si propone di analizzare le modalità con cui gli LLM vengono integrati nel processo di analisi, il loro impatto sul flusso di lavoro dei reverse engineer e la percezione del loro utilizzo.

Per lo svolgimento dello studio, i ricercatori hanno selezionato 48 partecipanti, equamente suddivisi tra reverse engineer con esperienza limitata ed esperti. I partecipanti sono stati quindi coinvolti in un esperimento controllato, nel quale dovevano analizzare due binari progettati in stile *Capture The Flag* (CTF), contenenti sfide rappresentative di comuni attività di SRE. L'esperimento prevedeva due condizioni, rispettivamente con e senza il supporto di un LLM (l'assistenza LLM veniva fornita attraverso un'interfaccia integrata nel flusso di lavoro).

L'osservazione delle attività svolte dai partecipanti ha portato all'individuazione di 18 risultati sperimentali, successivamente sintetizzati dagli autori in cinque principali conclusioni [14]:

Supporto nelle fasi iniziali. L'uso dei LLM è particolarmente utile nelle fasi iniziali dell'analisi, grazie al fatto che fornisce una visione d'insieme a un basso costo cognitivo. Questo approccio avvantaggi soprattutto i reverse engineer meno esperti, mentre il beneficio si riduce nel caso di analisi più approfondite.

Impatto limitato sugli esperti. L'apporto dei LLM migliora solo marginalmente le competenze degli esperti, i quali preferiscono delegargli le attività di routine, così da riallocare il proprio impegno sulla comprensione di logiche complesse, redistribuendo così lo sforzo.

Rischio delle allucinazioni. Anche delle allucinazioni rare possono compromettere seriamente il flusso di analisi, visto che la formulazione e l'indagine di ipotesi errate prolunga i tempi complessivi di lavoro.

Recupero degli artefatti vs. comprensione semantica. La comprensione semantica del programma da parte del reverse engineer dipende fortemente dal recupero degli artefatti. Nonostante i LLM aiutino a recuperare tipi, nomi di funzioni e variabili, questo non si traduce automaticamente in una comprensione profonda del funzionamento del codice.

Limiti nella delegazione delle attività. Ai LLM possono essere demandate solo le attività più semplici, come ad esempio la descrizione e la caratterizzazione di funzioni *single-purpose* e di algoritmi standard. Nel caso di funzioni complesse, con più ruoli o dotate di logiche intricate è necessaria un'analisi umana iterativa, eventualmente supportata dai LLM.

Lo studio evidenzia dunque che, in assenza di un avanzamento tecnologico nell'ambito della GenAI, di un miglioramento delle conoscenze specifiche dei LLM e, soprattutto, di un'evoluzione delle modalità di interazione, questi modelli rimarranno per la maggior parte degli strumenti di supporto per principianti, piuttosto che veri collaboratori alla pari per gli esperti di SRE [14]. Tuttavia, è fondamentale sottolineare

che tale conclusione è stata dedotta considerando un paradigma di interazione basato su assistenti conversazionale e sullo stato dell'arte della GenAI precedente alle più recenti evoluzioni.

In questo conteso, l'emergere e la diffusione, tra il 2025 e il 2026, degli agenti AI basati su LLM ha introdotto un modello di interazione alternativo rispetto a quello tradizionale. Tali sistemi mirano a superare il ruolo di mero strumento di sintesi preliminare identificato dagli autori, avvicinandosi a forme di collaborazione uomo-LLM decisamente più profonde. Questo nuovo paradigma si caratterizza per la capacità di pianificare, iterare sulle ipotesi, utilizzare gli strumenti di supporto e agire autonomamente.

Nonostante ciò, al momento le evidenze empiriche sul raggiungimento di un ruolo di collaboratore o persino di una sostituzione completa del reverse engineer negli scenari complessi di SRE risultano ancora limitate. Di conseguenza, l'effetto di questi miglioramenti tecnologici deve ancora essere pienamente inquadrato e validato, in particolare nello specifico ambito degli agenti AI autonomi.

3

Il problema

Questo capitolo ha l'obiettivo di introdurre il problema affrontato nella tesi. Dopo una prima definizione generale, verranno analizzati in dettaglio gli input e gli output attesi dal sistema, evidenziandone le principali caratteristiche. Successivamente, saranno identificati i requisiti e i vincoli progettuali che guideranno lo sviluppo della soluzione. Il capitolo si concluderà con una rappresentazione sintetica del sistema con un diagramma di contesto.

3.1 Definizione del problema

Il problema oggetto di studio e sviluppo in questa tesi consiste nella progettazione e nella successiva implementazione di un sistema in grado di ricevere in ingresso un file binario arbitrario e, qualora questo appartenga a una botnet, generare automaticamente un modulo software capace di catturare le comunicazioni Command-and-Control ricevute dall'infrastruttura di comando e di tradurle in una rappresentazione strutturata.

La motivazione principale alla base di questo lavoro consiste nell'automatizzare e velocizzare, ove possibile, le attività richieste di malware analysis e produzione del codice. Quando svolte manualmente da un esperto di cybersecurity, tali attività richiedono competenze specialistiche, un grande investimento di tempo e un significativo impegno operativo.

La responsabilità primaria del sistema consiste pertanto nell'analizzare in maniera autonoma campioni di malware appartenenti a diverse botnet, al fine di ricavare tutte le informazioni necessarie alla generazione di un modulo di cattura. Tale modulo dovrà soddisfare sia i requisiti derivanti dalla definizione iniziale del problema, sia quelli che verranno progressivamente individuati e formalizzati nel corso del presente capitolo e delle successive fasi di progettazione e implementazione.

Il processo di analisi dovrà quindi essere orientato a raggiungere l'obiettivo finale di generazione del modulo, concentrandosi sull'acquisizione delle conoscenze strettamente necessarie a ricostruire il protocollo di comunicazione utilizzato. Di conseguenza, tra le responsabilità del sistema non rientra la comprensione completa delle caratteristiche e del funzionamento interno del malware analizzato. Nonostante il processo di analisi possa, come effetto collaterale, portare alla ricostruzione di ulteriori dettagli sul comportamento del campione, tali informazioni non costituiscono un obiettivo del sistema.

Analogamente, il sistema non avrà il compito di verificare o garantire il corretto funzionamento del modulo prodotto e quindi l'analisi si concluderà con la generazione del modulo stesso. La successiva validazione del modulo e la valutazione dei risultati ottenuti resteranno invece di competenza dell'utente finale. Quest'ultimo potrà utilizzare il modulo generato come preferisce, potendo modificarlo, estenderlo, fino ad arrivare a eseguirlo.

3.2 Input e output del sistema

3.2.1 Input

Il sistema dovrà essere progettato per accettare come unico dato di ingresso un file binario arbitrario. Nella pratica operativa, tale file corrisponderà generalmente a un campione di malware in formato eseguibile appartenente a una botnet; tuttavia, il sistema non dovrà imporre alcun vincolo sulla natura o sul formato del file fornito.

Un requisito fondamentale dell'architettura è la capacità di operare in completa autonomia, senza disporre di alcuna conoscenza preliminare sul campione analizzato. Di conseguenza, al sistema non potranno essere fornite informazioni aggiuntive relative al file di ingresso, quali, ad esempio, la famiglia di appartenenza, descrizioni del comportamento o delle funzionalità del malware, oppure il protocollo di comunicazione utilizzato.

3.2.2 Output

La natura dell'output che verrà restituito dal sistema dipenderà strettamente dall'esito dell'eventuale fase di analisi preliminare effettuata sul file ricevuto in ingresso.

Nel caso in cui il sistema determini con certezza che il file analizzato non sia malevolo, oppure che, pur essendolo, la sua analisi non sia ritenuta opportuna o di utile interesse, il processo verrà interrotto e il file sarà scartato.

Al contrario, qualora il file di input sia identificato come un campione malevolo degno di attenzione, il sistema dovrà svolgere tutte le attività necessarie per arrivare a costruire un modulo di cattura sviluppato ad hoc sulle caratteristiche del malware in esame.

Il modulo generato dovrà essere implementato nel linguaggio Python e dovrà essere *self-contained*, ovvero completamente autoconsistente. Il codice prodotto dovrà contenere al suo interno tutte le definizioni, le classi e le dipendenze necessarie per implementare autonomamente i metodi richiesti per la cattura del traffico, l'elaborazione e il parsing dei comandi C&C e, più in generale, tutte le funzionalità previste dal modulo.

La scelta di utilizzare Python come linguaggio di destinazione è dovuta a diversi aspetti, tra cui due in particolare. Il primo riguarda il fatto che tale linguaggio è ampiamente utilizzato nell'ambito della cybersecurity e della *Network Forensics*, grazie alla disponibilità di un ampio ecosistema di librerie e strumenti dedicati all'analisi e all'elaborazione dei dati di rete [15].

Il secondo aspetto riguarda un requisito tecnico legato alla gestione del traffico di rete in tempo reale. L'impiego di costrutti avanzati di Python, come i generatori, attraverso l'istruzione `yield`, consente di elaborare progressivamente i messaggi C&C secondo un modello *lazy*, anziché mantenerne l'intero

contenuto in memoria [16]. Tale approccio permette infatti di limitare il consumo di memoria del modulo, facilitandone l'integrazione con altri sistemi e rendendolo idoneo a operare per periodi di tempo prolungati.

3.3 Requisiti del problema

Una volta definiti il problema, gli obiettivi del sistema e le caratteristiche dell'input e dell'output attesi, la progettazione dell'architettura gode di un ampio margine di libertà. In linea di principio, non sono infatti imposti specifici vincoli riguardo agli strumenti, alle tecniche o ai componenti da impiegare per la realizzazione della soluzione.

Tuttavia, tale libertà progettuale non è assoluta, poiché ogni soluzione proposta dovrà soddisfare un gruppo di requisiti che dovrà essere preso in considerazione durante l'intero processo di progettazione e implementazione del sistema.

Per chiarezza espositiva, tali requisiti possono essere suddivisi in due categorie principali: requisiti funzionali e requisiti di sicurezza.

Requisiti funzionali.

- **Indipendenza dalla famiglia di botnet.**

Il sistema dovrà essere progettato in modo da poter essere applicato a campioni appartenenti a differenti famiglie di botnet, senza introdurre dipendenze specifiche da una particolare tipologia o dalle caratteristiche peculiari di una singola famiglia.

Lo stesso principio di indipendenza dovrà essere applicato anche al protocollo di comunicazione C&C. La soluzione proposta dovrà essere in grado di gestire differenti modalità di interazione tra il malware e l'infrastruttura di C&C, indipendentemente dal protocollo di comunicazione, dalle tecnologie di trasporto o dai formati dei messaggi utilizzati.

- **Autonomia del processo di analisi.**

Il sistema dovrà operare in maniera completamente autonoma, senza richiedere interventi da parte dell'utente durante le diverse fasi dell'analisi. Un utente del sistema dovrà fornire esclusivamente il file binario in ingresso e, nel caso in cui questo appartenga a una botnet, si limiterà a ricevere il risultato prodotto dal sistema al termine del processo di analisi.

L'utente potrà poi intervenire solo a processo concluso, eseguendo il modulo generato o operando su di esso, apportando correzioni, adattamenti o miglioramenti. A tal fine, è fondamentale garantire un adeguato livello di trasparenza rispetto alle operazioni svolte dal sistema durante l'analisi, mantenendo traccia delle attività eseguite. Ciò consentirà di agevolare l'utente nella comprensione del processo seguito dal sistema e di semplificare le eventuali modifiche che vorrà apportare al modulo generato.

Requisiti di sicurezza.

- **Natura difensiva del modulo generato.**

Il modulo prodotto dal sistema dovrà avere esclusivamente finalità difensive e dovrà limitarsi a catturare passivamente le comunicazioni provenienti dal server C&C. Le funzionalità implementate dovranno essere circoscritte alla gestione della connessione e alle fasi di ricezione, decodifica, *parsing* e classificazione dei messaggi ricevuti. Non dovranno essere inclusi comportamenti offensivi, né meccanismi volti a eseguire attacchi o altre azioni malevole. Le uniche comunicazioni attive consentite saranno quelle strettamente necessarie all'instaurazione e al mantenimento della sessione con il server C&C.

- **Sicurezza del processo di analisi.**

Il processo di analisi del file di input dovrà essere progettato in modo tale da garantire la protezione dell'ambiente di esecuzione. L'elaborazione dei campioni potenzialmente malevoli non dovrà in alcun modo esporre il sistema di analisi, né eventuali sistemi esterni coinvolti, a rischi di compromissione o alterazione del loro funzionamento.

Oltre ai requisiti precedentemente individuati, la progettazione dell'architettura dovrà seguire i principi dell'ingegneria del software, preferendo soluzioni che garantiscano la qualità, la manutenibilità e l'estensibilità del sistema. Pur non essendo requisiti funzionali, queste proprietà condizioneranno le scelte di progettazione e implementazione.

In particolare, l'architettura proposta dovrà adottare un approccio modulare, favorendo una chiara separazione delle responsabilità tra i componenti del sistema. La soluzione dovrà inoltre essere progettata per potersi integrare facilmente con eventuali pipeline di analisi e framework già esistenti. Infine, dovranno essere previste adeguate funzionalità di *logging*, utili a tracciare le principali operazioni svolte dal sistema e a supportare le attività di monitoraggio e debugging.

3.4 Interfaccia del modulo di output

Al fine di consentire l'integrazione del sistema all'interno di workflow esterni e pipeline di automazione più ampie, è indispensabile che l'output prodotto rispetti alcuni vincoli strutturali e comportamentali predefiniti.

Per questo motivo, si è scelto di definire tali vincoli già durante la fase di analisi dei requisiti, anticipandoli rispetto alle successive fasi di progettazione e implementazione.

In particolare, la conformità dell'output si articola su due livelli distinti:

Interfaccia del modulo. `BotNetInterface`

Per rendere i moduli generati immediatamente utilizzabili da sistemi esterni, è necessario definire un'interfaccia standardizzata. In questo modo, qualsiasi componente che ne conosca la struttura potrà caricare e utilizzare direttamente il modulo prodotto, senza conoscerne i dettagli interni e senza richiedere alcun adattamento.

Per tale motivo, ogni modulo generato dal sistema dovrà implementare l'interfaccia astratta `BotNetInterface`. Tale interfaccia impone la presenza nel modulo di due metodi fondamentali:

- Metodo `connect`. Ha lo scopo di inizializzare o, eventualmente, ripristinare la connessione con il server C&C. Verrà eseguito all'avvio del modulo, prima dell'inizio dell'attività di cattura, e

successivamente ogni volta che il metodo `sniff` terminerà la propria esecuzione a causa della chiusura della connessione o di un errore non recuperabile.

- Metodo `sniff`. Ha lo scopo di mantenere attiva la connessione con il server C&C ed eseguire la cattura dei messaggi. Il metodo è progettato per rimanere in esecuzione indefinitamente e dovrà essere implementato come un generatore *Python*, restituendo mediante l'istruzione `yield` gli oggetti corrispondenti ai messaggi catturati, non appena questi sono stati ricevuti e decodificati.

L'interfaccia astratta che ogni modulo generato dovrà implementare è riportata in Figura 3.1.

```

1 from abc import ABC, abstractmethod
2 class BotNetInterface(ABC):
3     @abstractmethod
4     def connect(self):
5         pass
6     @abstractmethod
7     def sniff(self):
8         pass

```

Figura 3.1: Interfaccia astratta `BotNetInterface`

La Figura 3.2 mostra invece un possibile esempio di utilizzo pratico di un modulo conforme all'interfaccia definita.

```

1 b = BotNet()
2 while True:
3     try:
4         b.connect()
5         for message in b.sniff():
6             print(message)
7     except:
8         pass

```

Figura 3.2: Esempio di uso di un generico modulo `BotNet` che implementa `BotNetInterface`

Schema dei messaggi. MessageSchema

Considerando la natura eterogenea delle botnet che il sistema dovrà analizzare, appartenenti a famiglie differenti e caratterizzate da protocolli di comunicazione e formati dei messaggi diversi, risulta indispensabile adottare un modello dati unificato per rappresentare le informazioni catturate.

Per tale motivo, tutti gli oggetti restituiti dal metodo `sniff` dovranno essere conformi a uno schema comune, definito all'interno del file `MessageSchema`. La standardizzazione della rappresentazione dei messaggi costituisce un prerequisito essenziale affinché eventuali componenti successive possano elaborare i dati in modo uniforme, indipendentemente dalla botnet da cui essi provengono.

A differenza della `BotNetInterface`, per la quale vengono definiti già in questa fase i metodi e il comportamento atteso, l'analisi dei requisiti non impone alcun vincolo sulla struttura interna del `MessageSchema`. L'unico requisito consiste nell'esistenza di uno schema fisso e condiviso, la cui definizione effettiva potrà essere gestita direttamente nella fase di implementazione del sistema.

Pertanto, l'output prodotto dal sistema non dovrà più limitarsi a essere funzionalmente corretto, ma dovrà rispettare entrambi i vincoli architetturali descritti. La conformità a questi due nuovi requisiti permetterà infatti di assicurare l'interoperabilità, la riusabilità e la scalabilità del processo di cattura.

3.5 Diagramma di contesto del sistema

Considerando il sistema come una generica *black box*, la trattazione del problema può concludersi con la costruzione del relativo diagramma di contesto, riportato in Figura 3.3. Il diagramma di contesto fornisce una rappresentazione ad alto livello del sistema, dei suoi confini e delle principali interazioni con l'ambiente operativo, senza entrare nel dettaglio della sua struttura interna.

Da un lato, il sistema sviluppato si deve occupare dell'analisi del file binario di input e, qualora siano soddisfatte le condizioni previste, della generazione del relativo output. Dall'altro, viene rappresentato un possibile utilizzo operativo del modulo generato da parte di un utente, che può impiegarlo per la cattura del traffico dell'infrastruttura C&C e per la restituzione dei messaggi intercettati in forma strutturata.

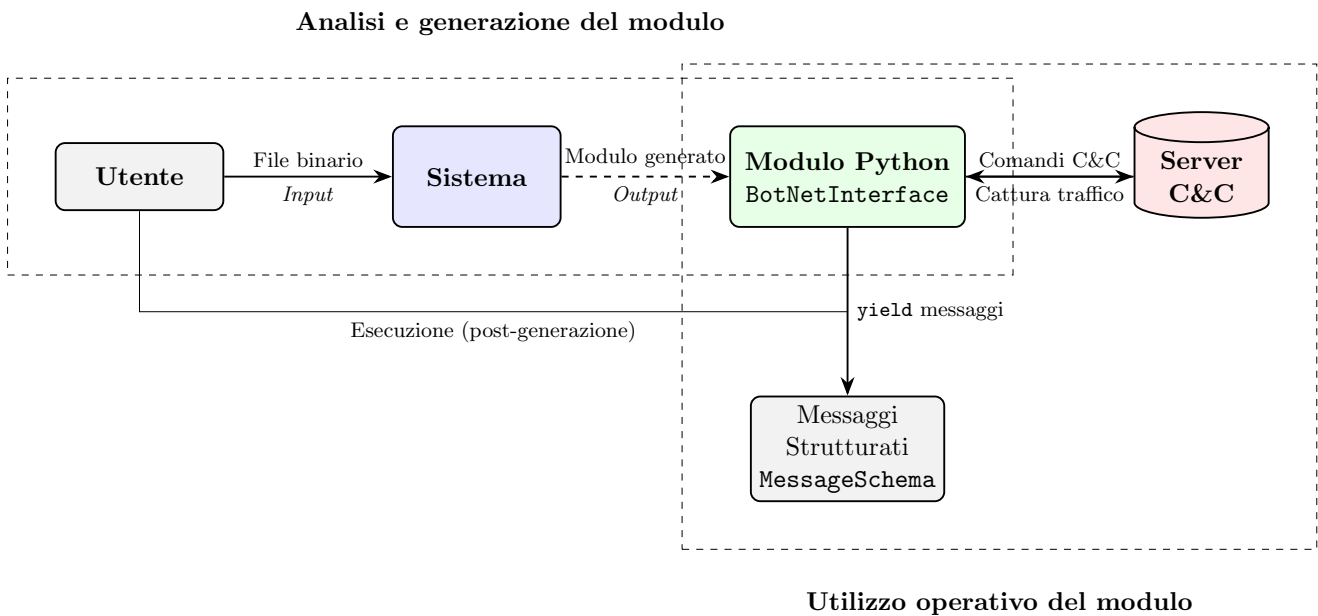


Figura 3.3: Diagramma di contesto del sistema

4

L'architettura proposta

Questo capitolo ha l'obiettivo di presentare l'architettura proposta per risolvere il problema affrontato dalla tesi. In primo luogo verrà mostrata l'architettura generale, utilizzando un diagramma dei componenti, e successivamente, per ciascun componente, verranno individuate le caratteristiche principali. Infine, verrà descritto il funzionamento del sistema da un ulteriore punto di vista, illustrando un flusso di esecuzione completo mediante un diagramma delle attività e approfondendo il meccanismo di *tool calling* con un diagramma di sequenza.

4.1 Architettura del sistema

Una modalità efficace per illustrare l'architettura proposta per il sistema e fornire una visione complessiva della sua organizzazione consiste nel ricorrere al diagramma dei componenti. Il diagramma UML dei componenti è un diagramma strutturale utilizzato per rappresentare la struttura di un sistema software attraverso i suoi principali componenti e le relazioni che intercorrono tra essi. Esso consente di descrivere l'organizzazione delle diverse parti del sistema e le modalità attraverso cui queste collaborano per realizzare le funzionalità complessive [17].

Il diagramma dei componenti realizzato è riportato in Figura 4.1.

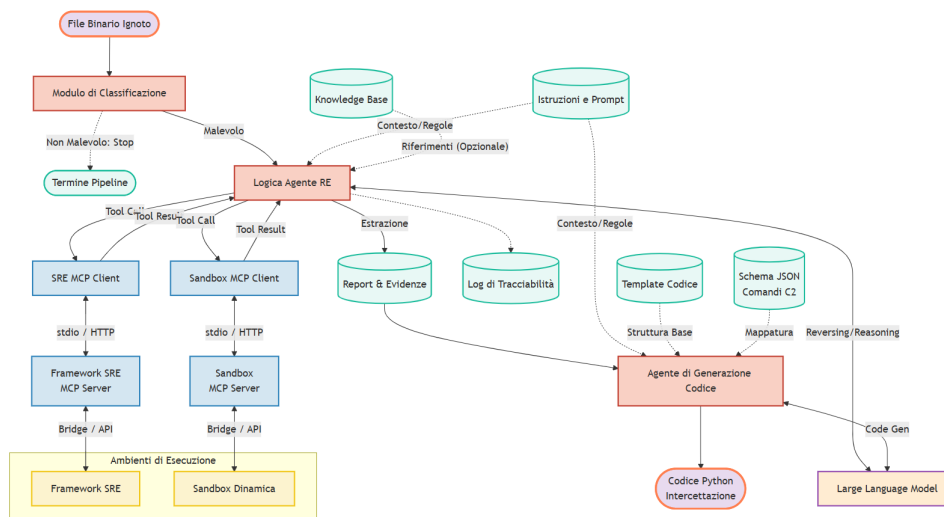


Figura 4.1: Diagramma UML dei componenti dell'architettura proposta

Questo diagramma permette quindi di mettere in evidenza le principali aree funzionali in cui è organizzato il sistema. In particolare, l'architettura è progettata come una *pipeline* di analisi automatizzata, suddivisa in moduli, all'interno della quale si distinguono, da un lato, la logica decisionale e gli agenti basati su LLM e, dall'altro, gli ambienti di esecuzione esterni con cui questi interagiscono attraverso il Model Context Protocol. A queste componenti si affiancano inoltre gli artefatti di input e di output, che comprendono sia l'input e l'output complessivi del sistema, sia i contenuti forniti alle diverse fasi della pipeline e i risultati prodotti nel corso dell'elaborazione.

4.2 Componenti del sistema

In questa sezione vengono descritti nel dettaglio i singoli componenti che costituiscono l'architettura proposta. Per ciascuno di essi vengono individuati i principali requisiti, le responsabilità e le funzionalità messe a disposizione degli altri componenti del sistema. La descrizione segue l'organizzazione mostrata nel diagramma di Figura 4.1, in ordine di appartenenza alle diverse aree funzionali individuate.

4.2.1 Artefatti

BINARY FILE – PYTHON MODULE. Rappresentano rispettivamente l'input e l'output principale del sistema. I relativi requisiti e le caratteristiche coincidono con quelli già descritti nel Capitolo 3 e non richiedono, a livello progettuale, ulteriori modifiche o integrazioni.

BotNetInterface – MessageSchema. Come i precedenti, anche questi artefatti sono già stati introdotti nel Capitolo 3 e non richiedono modifiche a livello progettuale. L'interfaccia **BotNetInterface** mantiene pertanto la definizione precedentemente descritta, mentre la struttura concreta dei messaggi rappresentati da **MessageSchema** potrà essere definita durante l'implementazione.

PROMPT ARCHIVE. Raccoglie le istruzioni statiche che devono essere fornite ai due agenti, **SRE AGENT** e **CODE AGENT**, per definire le attività da svolgere. Esso comprende sia i *system prompt* che gli *user prompt* utilizzati.

Il *system prompt* è un insieme di istruzioni fornite al LLM che permette di definire il ruolo dell'agente, il comportamento da assumere, il tono da utilizzare, i vincoli e le capacità previste per l'applicazione specifica. Mentre il *system prompt* rimane fisso, gli *user prompt* cambiano a ogni interazione e rappresentano le attività specifiche che l'utente richiede al modello o all'agente di eseguire nel ruolo definito. Gli *user prompt* dovranno quindi descrivere nello specifico tali attività, indicando anche l'output atteso e il formato desiderato, oltre a eventuali informazioni aggiuntive.

KNOWLEDGE BASE. A livello progettuale, è definita come un elemento opzionale del sistema, senza imporre a priori una forma o una struttura interna. Il suo scopo è fornire al LLM un contesto informativo aggiuntivo, non necessariamente richiesto esplicitamente tramite tool, che possa supportare e migliorare l'attività di analisi. Tale contesto può comprendere, ad esempio, documentazione tecnica, istruzioni operative specifiche, informazioni relative a una determinata famiglia di malware o altre conoscenze ritenute rilevanti.

A differenza dei prompt, che tendono a rimanere relativamente stabili nel tempo, la KNOWLEDGE BASE è pensata per avere una natura maggiormente dinamica, potendo essere aggiornata o modificata in funzione delle esigenze del reverse engineer e dell'evoluzione delle conoscenze disponibili. Proprio per questa caratteristica, un'implementazione efficace potrebbe prevedere la raccolta di informazioni specifiche per le diverse famiglie di malware, che il SRE AGENT può utilizzare in funzione del singolo campione analizzato, evitando di incorporare direttamente nei prompt informazioni che possono variare da un campione all'altro.

SANDBOX FILES. Sono i file prodotti dal LLM all'interno dell'ambiente SANDBOX durante le attività di analisi svolte nel SRE AGENT. Tali file devono essere considerati *untrusted*, in quanto prodotti da un LLM che sta interagendo con un malware.

Per questo motivo, l'esecuzione automatica di tali file non deve essere consentita nel resto della pipeline. L'unica operazione automatica permessa è la lettura in forma testuale dei file con estensioni considerate attendibili, mentre qualsiasi altra operazione può essere eseguita esclusivamente dall'utente.

SRE REPORT. Rappresenta l'artefatto chiave dell'intera pipeline, poiché si colloca tra i due agenti e per questo deve rispondere a due scopi principali. Il primo consiste nel sintetizzare con un elevato livello di dettaglio l'attività di reverse engineering svolta dal SRE AGENT e, allo stesso tempo, nel fungere da *single source of truth* (SSOT) per la successiva produzione del PYTHON MODULE da parte del CODE AGENT. Il secondo consiste nel fornire una documentazione analizzabile, consultabile e verificabile dall'utente, che possa quindi validare autonomamente quanto rilevato e correggere eventuali errori.

A questo scopo, è importante che il file, o eventualmente l'insieme di file, utilizzi una struttura chiara, magari adottando un formato semistutturato (ad esempio Markdown), che ne faciliti sia la lettura da parte dell'umano che la comprensione da parte del LLM. Il contenuto deve inoltre utilizzare un linguaggio appropriato e riportare in dettaglio i risultati dell'analisi, fornendo una chiara sequenza di evidenze e riferimenti specifici a supporto delle conclusioni, quali indirizzi di memoria e nomi utilizzati.

LOGGING FILES. Permettono di tracciare in dettaglio le azioni svolte dal LLM all'interno del SRE AGENT. Possono consistere in uno o più file, caratterizzati da un diverso grado di strutturazione, e includere, ove previsto, anche informazioni relative all'utilizzo delle risorse computazionali e ai tempi di esecuzione.

Il loro scopo principale è consentire all'utente di effettuare un'analisi più approfondita delle attività del LLM rispetto a quella possibile attraverso il solo SRE REPORT, oltre a supportare le attività di debugging e, nei casi previsti, permettere di riprendere o continuare manualmente l'esecuzione a partire da uno stato precedente.

CODE TEMPLATES. Sono costituiti da uno o più file che il LLM del CODE AGENT può utilizzare come riferimento durante la generazione del PYTHON MODULE. Tra questi deve essere presente almeno un file Python principale contenente una classe che implementi l'interfaccia `BotNetInterface`. A

tale classe o file ne possono essere affiancati altri, con l'obiettivo di modularizzare il codice e mantenere separate le responsabilità, ad esempio attraverso moduli specifici dedicati alla classificazione e al parsing dei messaggi. Il template può essere unico oppure, analogamente alla KNOWLEDGE BASE, può essere declinato in versioni specifiche per ciascuna famiglia di malware.

L'introduzione dei template di codice risponde a due obiettivi. Il primo consiste nel standardizzare e strutturare, almeno in parte, l'output finale (PYTHON MODULE), così da semplificarne la valutazione e la revisione da parte dell'utente prima di un'eventuale esecuzione. In particolare, i template forniscono una struttura e una logica comuni, limitando la variabilità del codice prodotto dal LLM anche in presenza di campioni simili, pur lasciando una certa libertà nella sua implementazione. Il secondo obiettivo consiste nel ridurre la possibilità che il codice finale introduca funzionalità malevole, fornendo al LLM un'implementazione di riferimento simile a quella attesa, che per l'appunto non include funzionalità di questo tipo.

4.2.2 Componenti decisionali

LARGE LANGUAGE MODEL. Costituisce il fulcro centrale dei due agenti e agisce come motore cognitivo, fornendo le capacità necessarie alla comprensione, al ragionamento, alla pianificazione e, ove previsto, all'utilizzo dei tool. Viene utilizzato come base per implementare le funzionalità previste sia dal SRE AGENT sia dal CODE AGENT.

Dal punto di vista tecnico, il modello deve disporre di dimensioni e prestazioni sufficienti a supportare le attività previste, di adeguate capacità di *reasoning*, particolarmente rilevanti per questo tipo di attività, e della capacità nativa di emettere tool call strutturate, anche in parallelo, nonché di una finestra di contesto sufficientemente ampia.

CLASSIFIER. È un modulo che agisce come *gatekeeper* della pipeline di analisi e ha il compito di esaminare preliminarmente il BINARY FILE ricevuto, decidendo se instradarlo verso il SRE AGENT oppure interrompere prematuramente il flusso di analisi, così da evitare l'impiego di risorse nello studio di campioni che non lo richiedono o che non risultano interessanti.

Inoltre, pur non essendo necessario per il funzionamento di base, il modulo può essere esteso per estrarre, quando possibile, ulteriori informazioni preliminari, come l'identificazione della famiglia di botnet a cui appartiene il campione. Tali informazioni possono essere utilizzate per fornire al SRE AGENT un contesto più specifico, attraverso la KNOWLEDGE BASE, anche se il SRE AGENT deve comunque essere in grado di svolgere l'attività di analisi anche senza di esse.

SRE AGENT. Si occupa di orchestrare l'attività di SRE sul campione di malware ricevuto. Utilizzando il LLM, a partire dal BINARY FILE, pianifica la strategia di analisi, tenendo conto dei prompt e, ove disponibile, della *Knowledge Base*, e la mette in atto attraverso il collegamento, mediante il protocollo MCP, ai tool esposti dagli ambienti di esecuzione.

L'agente deve gestire le tool call generate dal modello, inoltrandole ai rispettivi MCP Client, e fornire al modello i risultati delle operazioni eseguite. Infine, al termine dell'analisi, deve raccogliere le informazioni ottenute e sintetizzarle all'interno del SRE REPORT. Il SRE AGENT deve inoltre

occuparsi delle funzionalità di logging richieste e adattare il proprio comportamento in funzione del fatto che il campione analizzato sia *stripped* o *non stripped*.

CODE AGENT. Utilizza il LLM per guidare e coordinare le attività di generazione del codice, sulla base dei prompt che ne definiscono le istruzioni operative. Riceve come input le informazioni prodotte durante la fase di analisi e contenute nel *SRE Report*, utilizzando inoltre come riferimento lo schema dei messaggi **MessageSchema** e i **CODE TEMPLATES**, tra cui c'è anche l'interfaccia **BotNetInterface**.

A partire da tali elementi, l'agente deve estrarre le informazioni necessarie, combinarle e utilizzarle per generare il *Python Module* di cattura. L'agente è pertanto responsabile di produrre un modulo funzionante e conforme ai formati e alle strutture richieste, in particolare rispettando l'interfaccia prevista e senza introdurre alcuna capacità offensiva.

4.2.3 Integrazione MCP

SRE MCP Server – Sandbox MCP Server. Ciascun server è dedicato al rispettivo ambiente di esecuzione e deve esporre i *tools* che potranno essere utilizzati dal **SRE AGENT**, in conformità al protocollo MCP. Ogni *tool* deve avere un *name* univoco e che permette di identificare chiaramente il proprio scopo. Ad esso sono associati inoltre un *title*, opzionale, una *description*, che specifica il contesto e le modalità di utilizzo, e infine un *inputSchema*, un JSON Schema che specifica, per ciascun parametro di input, il tipo, la relativa descrizione e se questo è obbligatorio (*required*) o opzionale [9].

Il server deve inoltre predisporre il meccanismo di trasporto attraverso cui accettare le connessioni dei client, utilizzando il protocollo **stdio** nel caso di client locali oppure **Streamable HTTP** nel caso di client remoti. A fronte delle richieste di esecuzione dei *tool* ricevute dai client, il server deve effettuare il *dispatch* verso la relativa implementazione, che può essere realizzata direttamente oppure indirettamente, delegando l'operazione effettiva a un altro componente. Dopo aver eseguito il *tool*, il server deve restituirne il risultato, eventualmente dopo averlo opportunamente formattato.

SRE MCP Client – Sandbox MCP Client. Ciascun client è associato al rispettivo server, locale o remoto. Utilizzando l'*endpoint tools/list* del server MCP, deve esporre una funzionalità che permetta al **SRE AGENT** di scoprire i *tool* disponibili. Analogamente, mediante l'*endpoint tools/call*, deve esporre una funzionalità che permetta di invocare un *tool*, specificandone il *name*, una stringa, e gli eventuali **arguments**, rappresentati da un oggetto JSON.

Dal momento che l'utilizzatore di tali funzionalità sarà il LLM all'interno del **SRE AGENT**, può inoltre essere utile prevedere due funzionalità di parsing personalizzate. La prima riguarda la conversione della descrizione dei *tool* disponibili dal formato di schema standard MCP al formato accettato dal LLM. La seconda, eventualmente, può riguardare la formattazione specifica di alcune risposte ricevute dai *tool*.

4.2.4 Ambienti di esecuzione

SRE Framework. Deve fornire almeno le capacità necessarie per consentire lo svolgimento delle attività di analisi statica del codice ed eventualmente anche capacità di analisi dinamica avanzata. A questo scopo, è utile che disponga di funzionalità di disassemblaggio, decompilazione ed, eventualmente, debugging, oltre che di ulteriori strumenti di SRE. Tra le funzionalità di analisi statica da includere rientrano, ad esempio, l'analisi e la gestione delle funzioni e delle relative *cross-reference*, degli indirizzi e della memoria, del *type system*, nonché la gestione e la rinomina dei simboli.

Dal punto di vista dell'integrazione, è fondamentale che tali funzionalità siano esposte attraverso un'interfaccia, ad esempio mediante plugin o API, che ne permetta l'accesso da parte del SRE AGENT mediante il protocollo MCP, consentendogli di eseguire le azioni selezionate dal LLM.

Sandbox. Ha lo scopo di aggiungere funzionalità dinamiche per l'analisi, potenzialmente diverse da quelle offerte dalle classiche attività di esecuzione o debugging, così da poter essere affiancata e combinata al SRE FRAMEWORK.

L'obiettivo è fornire un ambiente generale nel quale, durante il processo di SRE, sia possibile costruire ed eseguire strumenti personalizzati per trasformare, analizzare e interpretare le informazioni estratte durante l'analisi, oppure per automatizzare determinate attività. La sandbox deve inoltre permettere di operare a livello di file, effettuando operazioni di lettura e scrittura secondo le necessità.

Uno degli aspetti più critici di questo componente consiste nel garantire la sicurezza dell'ambiente durante la sua implementazione. La sandbox deve garantire l'isolamento dall'host e da eventuali altre esecuzioni, operare con privilegi minimi e prevedere limiti alle risorse computazionali disponibili. L'accesso alla rete dovrebbe essere negato per impostazione predefinita e consentito esclusivamente qualora l'implementazione della sandbox garantisca adeguate condizioni di sicurezza.

Ai fini dell'integrazione con il SRE AGENT tramite il protocollo MCP, valgono considerazioni analoghe a quelle precedentemente illustrate per il SRE FRAMEWORK.

4.3 Flusso di esecuzione

Dopo aver descritto la struttura complessiva dell'architettura e le responsabilità dei singoli componenti, questa sezione presenta un esempio di flusso di esecuzione completo, illustrando il percorso seguito attraverso i diversi componenti del sistema da un generico BINARY FILE di input.

A questo scopo viene utilizzato un diagramma UML delle attività, un diagramma comportamentale che permette di rappresentare lo svolgimento di un processo attraverso il flusso delle attività, le decisioni, i possibili percorsi alternativi, le attività eseguite in parallelo e, ove necessario, il flusso degli oggetti [17].

Il diagramma delle attività realizzato è riportato in Figura 4.2. A differenza del diagramma dei componenti presentato nella sezione precedente, questo diagramma ha una finalità esclusivamente descrittiva: rappresenta infatti un flusso semplificato utilizzato solamente per illustrare il comportamento del sistema, senza costituire una specifica vincolante per l'implementazione.

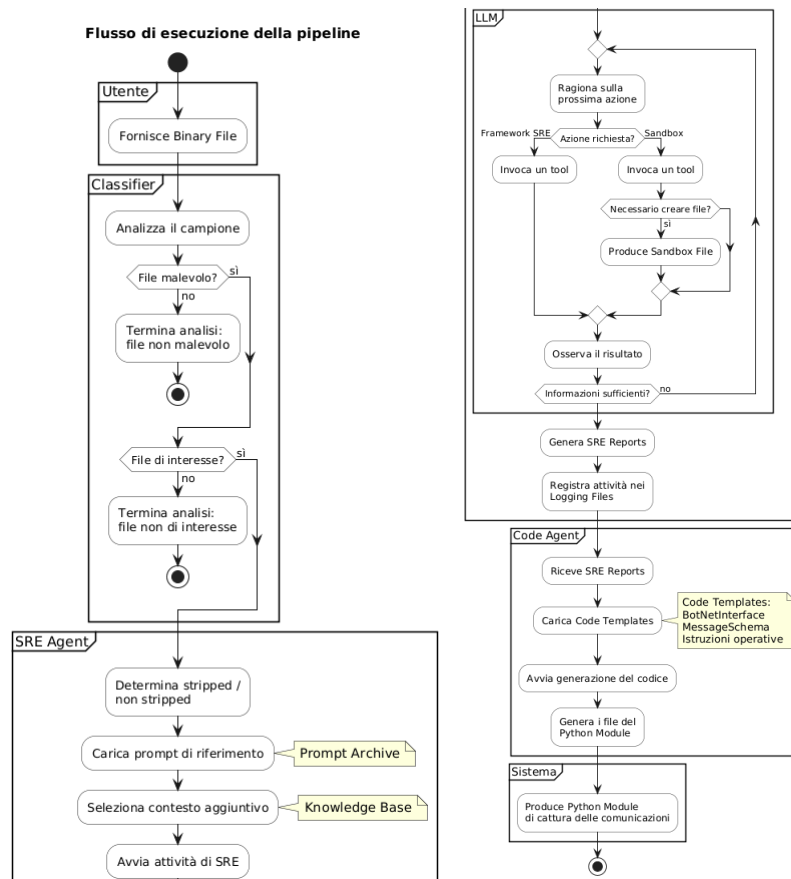


Figura 4.2: Diagramma UML delle attività della pipeline di analisi

Il flusso di esecuzione ha inizio quando l'utente fornisce in ingresso al sistema il `BINARY FILE`. Quindi, il `CLASSIFIER` ne esegue un'analisi preliminare, con lo scopo di determinare se il campione sia malevolo oppure no. Quando è possibile stabilire con certezza che il campione non è malevolo oppure che, pur essendo malevolo, non è di interesse per il sistema, allora la pipeline di analisi viene interrotta. Negli altri casi il campione viene trasferito dal `CLASSIFIER` al `SRE AGENT`.

Come prima cosa, il `SRE AGENT`, basandosi sulla presenza dei simboli, determina se il campione è non stripped o stripped. Nel secondo caso, l'agente potrebbe subito svolgere qualche azione apposita oppure potrà adattare di conseguenza il proprio comportamento direttamente durante lo svolgimento delle attività di SRE. A questo punto l'agente carica dal `PROMPT ARCHIVE` i prompt di riferimento e, quando possibile, utilizza le informazioni rese disponibili dal `CLASSIFIER` per selezionare dalla `KNOWLEDGE BASE` il contesto aggiuntivo pertinente al campione.

La fase centrale è dunque costituita dall'attività di SRE svolta dal `SRE AGENT` attraverso il `LLM`, secondo un ciclo di iterativo di ragionamento, azione e osservazione. Per fare ciò, il modello utilizza i tool messi a disposizione dagli ambienti di esecuzione `FRAMEWORK SRE` e `SANDBOX`, interagendoci secondo il protocollo `MCP`. Nel caso dell'ambiente `SANDBOX`, il `LLM` potrà anche produrre e salvare dei `SANDBOX FILES`. Quando l'agente dispone di informazioni sufficienti, conclude l'analisi, producendo uno o più file riassuntivi (`SRE REPORTS`), mentre l'elenco completo delle attività svolte viene registrato nei relativi `LOGGING FILES`.

Infine, i `SRE REPORTS` vengono passati al `CODE AGENT`, che carica i `CODE TEMPLATES`, che comprendono la `BotNetInterface`, il `MessageSchema` e le istruzioni operative dal `PROMPT ARCHIVE`.

Il LLM dell'agente utilizza questo insieme di informazioni per generare i file necessari a costruire il PYTHON MODULE di cattura delle comunicazioni, che corrisponde all'output finale del sistema.

4.4 Tool calling

Considerata la rilevanza del meccanismo di *tool calling* nell'architettura proposta e la frequenza con cui viene coinvolto durante l'attività di SRE, risulta utile fornire una rappresentazione più dettagliata delle interazioni che avvengono tra il SRE AGENT e gli ambienti di esecuzione attraverso il protocollo MCP. I soli diagrammi dei componenti e di attività non risultano infatti adeguati a rappresentare in modo chiaro ed efficace la natura di tali interazioni.

A tale scopo, si utilizza un diagramma UML di sequenza, un diagramma comportamentale che rappresenta come diversi componenti interagiscono tra loro nel corso del tempo, descrivendo lo scambio di messaggi relativo a una determinata interazione e mettendo in evidenza l'ordine temporale degli eventi [17].

Il diagramma di sequenza realizzato è riportato in Figura 4.3. Esso rappresenta una sequenza esemplificativa, corrispondente a un sottoinsieme delle interazioni che si verificano durante le attività di analisi da parte del SRE AGENT.

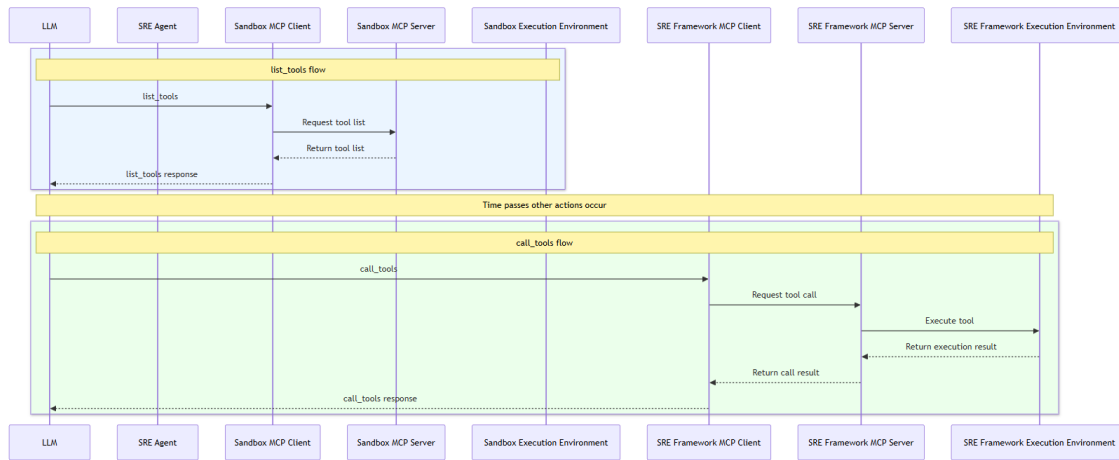


Figura 4.3: Diagramma UML di sequenza delle operazioni di tool calling

Il diagramma mostra nella prima parte della sequenza alcune delle operazioni preliminari relative alla creazione e alla configurazione degli oggetti coinvolti nelle interazioni successive, tra cui l'istanziamento dei MCP CLIENT e del LLM da parte del SRE AGENT.

La seconda parte della sequenza riguarda la scoperta dei tool disponibili per l'ambiente SANDBOX da parte dell'agente. Il SRE AGENT richiede al SANDBOX MCP CLIENT la lista dei tool tramite la funzione *list_tools()* e il client invoca quindi l'endpoint MCP *tools/list* del SANDBOX MCP SERVER, che restituisce la lista dei tool secondo il formato previsto dal protocollo MCP. Il client può eventualmente convertire tale formato in quello richiesto dal LLM, restituendo quindi la lista al SRE AGENT. Quest'ultimo può utilizzare la lista ottenuta come parametro per le richieste inviate al LLM, eventualmente limitandola anche a un sottoinsieme dei tool disponibili.

La terza parte riguarda invece l'effettiva chiamata di un generico tool del SRE FRAMEWORK. Il SRE AGENT, dopo aver invocato il LLM, riceve una risposta strutturata contenente una o più tool call. Il SRE AGENT estrae quindi le singole tool call e determina a quale ambiente di esecuzione appartengano. Nel caso considerato, la chiamata riguarda il SRE FRAMEWORK, per cui l'agente richiede al SRE MCP CLIENT l'esecuzione del tool tramite la funzione *call_tool(name, arguments)*. Il client invoca quindi l'endpoint `tools/call` del SRE MCP SERVER, specificando il *name* del tool e gli eventuali *arguments* come oggetto JSON. Il SRE MCP SERVER gestisce la richiesta eseguendo direttamente l'azione sul SRE FRAMEWORK oppure delegandola a un altro componente incaricato della sua esecuzione. Quando l'esecuzione dell'operazione richiesta termina, il SRE FRAMEWORK restituisce il relativo risultato al server, che può eventualmente formattarlo prima di restituirlo al MCP CLIENT. Quest'ultimo può a sua volta applicare un'ulteriore conversione del formato, prima di restituire il risultato al SRE AGENT. L'agente può infine inserire il risultato nel contesto del LLM e proseguire con l'attività di analisi.

5

L'implementazione

Questo capitolo ha l'obiettivo di trattare l'implementazione del sistema progettato. Verranno quindi discusse le scelte implementative effettuate e quindi descritto l'implementazione dei singoli componenti. Tutto ciò verrà fatto con diversi livelli di dettaglio, andando più nel profondo per cose interessanti/difficili, più brevemente invece per quelle banali.

5.1 Ambiente di sviluppo

L'implementazione del sistema è stata realizzata in Python 3.13 ¹, mentre per la gestione dell'ambiente di sviluppo e delle dipendenze del progetto è stato utilizzato Conda ².

Il motivo per cui la scelta del linguaggio di implementazione è ricaduta su Python è legato principalmente all'ampio ecosistema di librerie disponibili per le funzionalità richieste dal sistema. Tra queste rientrano, in particolare, librerie dedicate all'analisi del malware, all'integrazione con LLM e agenti, alle interazioni basate sul protocollo MCP e all'interazione con gli ambienti di esecuzione. La disponibilità di strumenti già consolidati ha pertanto consentito di concentrare lo sviluppo sulla logica specifica dell'architettura proposta, riducendo la necessità di implementare direttamente funzionalità di supporto.

5.2 Orchestrazione / Esecuzione

5.2.1 Classifier

Virus Total

Test malevolo e ipotesi famiglia.

YARA Rules

Conferma della famiglia.

¹<https://www.python.org>

²<https://docs.conda.io/projects/conda/en/latest/user-guide/tasks/index.html>

5.2.2 SRE Agent

- import in SRE Framework e analisi automatica (fun ghidra)
- determina con una funzione se stripped (fun ghidra)
- eventuale injection knowledge base
- doppio ciclo di renaming start/stop
- save (fun ghidra)
- doppio ciclo di reversing con due report

contatto con SRE framework e sandbox.

Knowledge Base

Selezione Tools Analisi Statica

Per questa implementazione è stato scelto solo un sottoinsieme di tool riguardanti analisi statica del codice. tre ragioni: - semplicità, cioè riduzione delle variabili in gioco e più controllo sulle azioni (eventualmente estendibile) - evitare di esporre all'esecuzione reale - the less mcp the better (LLM)

I tool scelti rientrano nelle seguenti categorie:

Selezione Tools Analisi Dinamica

Niente accesso al file e niente accesso a internet.

5.2.3 Code Agent

5.3 SRE Framework

Il framework di SRE adottato è Ghidra, sviluppato e mantenuto dal Research Directorate della National Security Agency (NSA) e distribuito come progetto open source secondo la Apache License 2.0.

L'ambiente Ghidra, *platform independent*, offre una suite ricca di strumenti e funzionalità per l'analisi binaria del codice compilato, tra cui disassembly, decompiling, graphing, debugging e scripting. Il framework supporta inoltre un'ampia varietà di *instruction set* relativi a differenti architetture di processore e numerosi formati eseguibili. Le attività di analisi possono essere svolte sia in modalità interattiva, attraverso una *Graphical User Interface* (GUI), sia in modalità automatizzata, rendendo Ghidra adatto all'integrazione all'interno di processi di analisi più complessi [4].

5.3.1 Ghidra

- Headless (no GUI)
- Docker Image
- 3

³<https://github.com/nationalsecurityagency/ghidra>

5.3.2 MCP Plugin

Per integrare le funzionalità di analisi offerte da Ghidra con il SRE AGENT in conformità con il protocollo MCP è stato utilizzato il software open source `ghidra-mcp`, disponibile sull'omonimo repository GitHub ⁴ e distribuito secondo la licenza Apache License 2.0.

Il software fornisce un'estensione per Ghidra e un insieme di componenti di supporto che permettono di rendere accessibili le funzionalità dell'ambiente di analisi attraverso il protocollo MCP, esponendo complessivamente più di duecento tool. Si tratta di una soluzione in continua evoluzione che supporta l'integrazione con Ghidra Server, la gestione di più progetti e file, l'esecuzione sia in modalità GUI sia in modalità headless e il deployment mediante Docker.

L'insieme delle funzionalità messe a disposizione è particolarmente ampio e comprende, innanzitutto, le principali operazioni di analisi binaria, relative a funzioni, data flow, strutture dati, stringhe, operazioni di importazione ed esportazione e accesso alla memoria. A queste si aggiungono funzionalità più avanzate, tra cui, nell'ambito dell'analisi dinamica, l'emulazione del P-code e l'integrazione con il debugger, oltre a strumenti per la gestione dei progetti, del controllo di versione, degli script di Ghidra. Sono inoltre supportate l'esecuzione batch delle operazioni e l'analisi di più programmi contemporaneamente.

5.3.3 Ghidra MCP Server

Il server MCP del software `ghidra-mcp` è implementato mediante il modulo `bridge_mcp_ghidra`, sviluppato in Python utilizzando la libreria FastMCP ⁵. Il modulo costituisce il livello intermedio tra i client MCP e l'estensione di Ghidra e supporta sia il trasporto *stdio*, sia il trasporto *Streamable HTTP*.

Una caratteristica interessante del modulo è l'organizzazione dei tool in gruppi semantici, che permette di gestire in maniera selettiva l'insieme delle funzionalità esposte al LLM. In modalità standard, il server rende disponibili tutti i gruppi di tool; tuttavia, considerata l'ampiezza del catalogo, il software può essere eseguito anche in modalità *lazy*, caricando inizialmente solo i gruppi di tool principali. Gli altri gruppi possono essere individuati e successivamente caricati o rimossi durante l'esecuzione da parte del LLM, in base alle necessità.

5.4 Ambiente Sandbox

La sandbox di SRE viene realizzata mediante un ambiente di esecuzione containerizzato, utilizzando Docker per la creazione, la configurazione e la gestione dei container.

La containerizzazione è una tecnica di isolamento a livello di sistema operativo che consente di eseguire un'applicazione e le relative dipendenze all'interno di un ambiente isolato. Rispetto alle macchine virtuali, che richiedono l'esecuzione di un intero sistema operativo guest, i container rappresentano un approccio più leggero per ottenere ambienti di esecuzione separati [18].

Nel contesto della SRE SANDBOX, Docker fornisce i meccanismi necessari alla creazione e alla configurazione dei container, consentendo di controllarne il filesystem, l'accesso alla rete e le risorse computazionali disponibili. Queste caratteristiche permettono di configurare l'ambiente in modo coerente con i requisiti di isolamento e sicurezza individuati durante la fase di progettazione.

⁴<https://github.com/bethington/ghidra-mcp>

⁵<https://gofastmcp.com/>

5.4.1 Configurazione

La configurazione dell'ambiente SRE SANDBOX è definita mediante un *Dockerfile*, che specifica l'immagine di base, le dipendenze e le impostazioni necessarie al funzionamento del container. L'immagine utilizzata è basata su Python 3.13 e Debian Slim ⁶, così da fornire un ambiente contenuto, ma sufficientemente completo da permettere al LLM di creare e modificare file, eseguire codice Python e impartire comandi tramite la shell.

A questo scopo, durante la fase di costruzione vengono installati gli strumenti di sistema e le librerie Python necessarie alle attività previste nella sandbox, tra cui quelle dedicate alle funzionalità crittografiche, come *cryptography* ⁷ e *pycryptodome* ⁸. Viene inoltre configurata una directory dedicata alla sandbox e viene creato un utente senza privilegi di root, utilizzato per l'esecuzione dei processi all'interno del container anziché l'utente root.

L'immagine così definita viene utilizzata per creare il container della sandbox, che implementa ulteriori misure di isolamento. In particolare, l'accesso alla rete viene completamente disabilitato, limitando l'interazione del codice con risorse esterne e confinando le operazioni alle sole funzionalità e dipendenze disponibili all'interno dell'ambiente. Per consentire lo scambio controllato dei SANDBOX FILES, una directory dell'host viene montata all'interno del container con permessi di lettura e scrittura. Infine, vengono imposti limiti alle risorse computazionali disponibili, in termini di memoria e capacità di elaborazione.

5.4.2 Sandbox MCP Server

Il server MCP della SANDBOX ha il compito di interagire con l'ambiente di esecuzione ed esporne le funzionalità attraverso tool MCP. Per la sua implementazione viene utilizzata la libreria Python *FastMCP* ⁹, mentre l'interazione con il container viene gestita attraverso la libreria *Docker SDK for Python* ¹⁰.

All'inizializzazione del server viene verificata la disponibilità del container della sandbox e, qualora questo non risulti in esecuzione, viene avviato automaticamente. Il server espone quindi un insieme di tool organizzati in due principali categorie:

File System I tool `list_files`, `read_file` e `write_file` permettono di consultare e modificare i file presenti nell'ambiente della sandbox. Ogni operazione prevede inoltre una gestione degli errori, così da fornire un'informazione esplicita in caso di esecuzione non riuscita.

Execution Il tool `execute_python_file` consente di eseguire file Python all'interno del container, imponendo un limite temporale all'esecuzione. Il tool `execute_command` permette invece di eseguire comandi della shell. Prima dell'esecuzione viene effettuato un controllo di sicurezza sui comandi richiesti per prevenire operazioni potenzialmente pericolose, come tentativi di privilege escalation, o apertura di nuove shell. In entrambi i casi, il risultato dell'esecuzione viene restituito in forma strutturata, includendo l'`exit code`, `stdout` e `stderr`.

⁶https://hub.docker.com/_/python

⁷<https://cryptography.io>

⁸<https://www.pycryptodome.org/>

⁹<https://gofastmcp.com/>

¹⁰<https://docker-py.readthedocs.io>

5.5 MCP Client

Per i due MCP client, `SANDBOX MCP CLIENT` e `GHIDRA MCP CLIENT`, è stata definita una classe di base comune, `BaseMCPClient`, responsabile dell'instaurazione e del mantenimento della connessione con il rispettivo MCP server mediante il metodo di trasporto *stdio*. Il ciclo di vita della connessione viene gestito attraverso un contesto asincrono, all'interno del quale viene inizializzata una `ClientSession` fornita dal *Python MCP SDK* ¹¹. La classe espone inoltre le due operazioni fondamentali per l'interazione con il server: la consultazione dei tool disponibili, tramite `list_tools`, e la loro invocazione, tramite `call_tool`.

5.5.1 LLM-MCP Client

La classe `BaseMCPClient` viene ulteriormente estesa da `LLMMCPClient`, che introduce il livello di integrazione con il LLM e si interpone tra la gestione della connessione MCP e le implementazioni specifiche dei singoli client. In particolare, i tool esposti dai server MCP vengono convertiti nel formato utilizzato dal meccanismo di *function calling* delle OpenAI API ¹², includendo le informazioni relative al nome, alla descrizione e ai parametri richiesti.

Entrambi i client specifici estendono quindi `LLMMCPClient`, ereditandone le funzionalità comuni. Poiché i server sono stati implementati secondo modalità di interazione uniformi e già predisposte all'utilizzo da parte di LLM, non è stato necessario introdurre ulteriori funzionalità, come una formattazione specifica per le risposte, nelle classi derivate.

A completamento di questa struttura, una classe dedicata fornisce un meccanismo per la registrazione dei client MCP e dei relativi tool. Essa consente di applicare opzionalmente ai nomi dei tool un *namespace*, costituito da un prefisso identificativo del client MCP, e di selezionare, specificandone i nomi, un sottoinsieme dei tool disponibili. In questo modo, i tool esclusi possono essere nascosti dall'insieme di strumenti esposti al LLM.

Nel complesso, queste componenti costituiscono un livello di astrazione comune per i diversi client MCP e ne semplificano l'integrazione con il LLM. I tool forniti dagli ambienti di esecuzione possono così essere esposti in modo uniforme e utilizzati attraverso il meccanismo di *tool calling* previsto per il SRE AGENT.

5.6 LLM

5.6.1 Server

Il LLM utilizzato dall'applicazione viene eseguito tramite un'istanza esterna di *vLLM* ¹³, un framework open source progettato per l'inferenza e il *serving* di LLM. Il server *vLLM* è quindi responsabile dell'esecuzione del modello e della gestione delle richieste di inferenza provenienti dall'applicazione.

L'interfaccia esposta da *vLLM* è compatibile con le API OpenAI ¹⁴, permettendo all'applicazione di interagire con il modello attraverso un'interfaccia standardizzata. Questa configurazione consente

¹¹<https://py.sdk.modelcontextprotocol.io/>

¹²<https://developers.openai.com/api/docs/guides/function-calling>

¹³<https://vllm.ai/>

¹⁴https://docs.vllm.ai/en/stable/serving/online_serving/openai_compatible_server/

di utilizzare modelli LLM eseguiti localmente, senza vincolare l'applicazione a uno specifico modello o fornitore esterno.

5.6.2 Client

Il LLM `Client` rappresenta il componente responsabile dell'interazione tra SRE AGENT/CODE AGENT e il LLM, integrando inoltre gli strumenti esposti attraverso i client MCP.

Alla sua inizializzazione vengono specificati i client MCP con i quali il LLM può interagire, quindi gli strumenti da essi esposti vengono raccolti e salvati in una struttura di risoluzione. Questa associazione permetterà successivamente di individuare, a partire dal nome del tool richiesto dal LLM, il client MCP responsabile della sua gestione. Il LLM `Client` si occupa quindi anche dell'invocazione dei tool richiesti, inoltrando ciascuna tool call al client MCP corrispondente e recuperandone il risultato.

Il componente permette inoltre di configurare i *sampling parameters* (parametri di campionamento), ovvero le variabili utilizzate per controllare le caratteristiche della generazione del testo, tra cui `temperature`, `top_p`, `top_k`, `min_p`, `presence_penalty` e `repetition_penalty`. È inoltre possibile controllare la modalità di ragionamento del LLM, abilitandola o disabilitandola tramite `preserve_thinking` e specificandone il livello mediante `reasoning_effort`.

Sono quindi messi a disposizione metodi per effettuare richieste singole al LLM mediante *chat completion*, secondo l'interfaccia prevista dalle OpenAI API ¹⁵ e dal relativo SDK Python ¹⁶. Le richieste possono essere effettuate sia con sia senza l'utilizzo dei tool; i parametri di campionamento, così come eventuali altri parametri supportati dal client, possono inoltre essere specificati o sovrascritti per la singola richiesta.

Infine, il componente fornisce la struttura di base dell'agente che viene utilizzata dal SRE AGENT. Questa è realizzata mediante un ciclo iterativo che gestisce autonomamente lo stato della conversazione, l'invocazione del LLM, l'esecuzione dei tool e l'inserimento nel contesto dei risultati ottenuti, proseguendo l'elaborazione fino al raggiungimento dell'obiettivo. L'implementazione segue il paradigma *ReAct*, alternando le fasi di ragionamento e di azione, preservando il ragionamento prodotto dal LLM, affidando al modello la selezione dei tool ed eseguendo sequenzialmente le tool call.

All'interno del ciclo di esecuzione non sono stati introdotti meccanismi di compattazione del contesto, così da mantenere la massima fedeltà alla cronologia dell'esecuzione e non perdere informazione. Sebbene questa scelta possa comportare il raggiungimento del limite della finestra di contesto, tale eventualità non si è rilevata problematica per gli utilizzi considerati nell'ambito di questa implementazione.

Logging

A supporto dell'esecuzione dell'agente, sono state implementate le funzionalità di logging individuate durante la fase di progettazione, secondo due modalità distinte e complementari.

La prima è orientata alla conservazione completa dell'esecuzione e produce un output strutturato in formato JSON, conforme al formato utilizzato dalle OpenAI API ¹⁷. Il log contiene l'intera cronologia della conversazione, includendo in particolare le tool call effettuate e i relativi risultati. La struttura

¹⁵<https://developers.openai.com/api/reference/resources/chat>

¹⁶<https://pypi.org/project/openai/>

¹⁷<https://developers.openai.com/api/reference/resources/chat>

adottata permette quindi di analizzare oppure importare e continuare a elaborare successivamente la singola esecuzione

La seconda modalità è invece orientata alla consultazione da parte dell'utente e presenta una sintesi leggibile dell'esecuzione. In particolare, vengono riportati il prompt iniziale e, per ciascun passo, alcune statistiche relative all'elaborazione (numero di token e tempo di esecuzione parziali), insieme al ragionamento e alla risposte prodotte dal LLM, oltre all'indicazione delle sole tool call effettuate. I risultati restituiti dai tool non vengono inclusi in questa visualizzazione, in modo da mantenere il focus sulla Chain of Thought sviluppata dal LLM.

5.7 MessageSchema

Come individuato già a partire dalla fase di analisi dei requisiti, è necessario definire uno schema comune per i messaggi restituiti dal `Python Module`. A questo scopo è stato adottato *JSON Schema*, particolarmente adatto a descrivere e verificare strutture dati rappresentate in formato JSON.

JSON Schema è un linguaggio dichiarativo progettato per descrivere la struttura e i vincoli dei dati rappresentati in formato JSON. Attraverso un insieme di parole chiave consente di specificare in modo chiaro le proprietà dei dati, con i tipi di dati consentiti, i campi obbligatori, i formati richiesti e altre restrizioni strutturali [19].

L'utilizzo di JSON Schema risulta particolarmente adatto per due motivi. Il primo è la possibilità di verificare la conformità di un documento JSON, denominato *instance*, rispetto a uno schema mediante un apposito *validator*, confrontando automaticamente la struttura e i contenuti dell'istanza con i vincoli definiti nello schema. Il secondo riguarda il suo utilizzo come forma di documentazione: lo schema fornisce infatti una descrizione esplicita della struttura e dei vincoli dei dati, comprensibile sia dallo sviluppatore, sia da un LLM.

5.7.1 Message Classification

Tra i costrutti messi a disposizione da JSON Schema rientra `oneOf`, che consente di specificare che un'istanza deve risultare valida rispetto a uno e un solo schema tra quelli definiti. Tale costrutto può essere sfruttato, nell'ambito dell'applicazione considerata, per distinguere semanticamente i messaggi catturati dall'infrastruttura di C&C in diverse tipologie. In questo modo, oltre a facilitare l'interpretazione dei messaggi da parte dell'utente, è possibile associare a ciascuna tipologia uno schema specifico, definendo ricorsivamente i campi appropriati in funzione della classificazione del messaggio.

Per l'implementazione considerata sono state pertanto individuate le seguenti quattro categorie:

ACK identifica i messaggi mediante i quali il server C&C richiede l'invio di un acknowledgment in risposta a un precedente messaggio.

Attack identifica i messaggi contenenti istruzioni relative all'esecuzione di un attacco.

Heartbeat identifica i messaggi utilizzati per il mantenimento della connessione e per il keepalive del traffico.

Command identifica i messaggi di servizio o generici che non rientrano nelle categorie precedenti, oppure i messaggi per i quali non si è in grado di effettuare una classificazione più specifica.

5.7.2 JSON Schema

Lo schema realizzato, conforme allo standard JSON Schema Draft 2020-12¹⁸, è riportato integralmente nelle appendici. Al primo livello dello schema vengono memorizzati il *timestamp* di ricezione del messaggio e l'indirizzo IP sorgente. Inoltre, è presente un campo dedicato alla classificazione del messaggio, secondo le tipologie introdotte nella sezione precedente, a cui segue uno schema annidato contenente le informazioni specifiche relative alla tipologia individuata.

Per ciascuna categoria sono quindi definiti vincoli e proprietà specifici. I messaggi **ACK** contengono l'informazione associata allo stato della richiesta di acknowledgment, mentre i messaggi **Heartbeat** riportano un numero di sequenza lato server e, opzionalmente, il tempo di connessione del bot. I messaggi *Command* includono invece il *payload* originale e, quando è possibile determinarla, una stringa che identifica la tipologia del comando. Infine, tutte le tipologie di messaggio prevedono un campo opzionale **notes**, destinato a contenere informazioni aggiuntive sotto forma di stringa oppure valorizzato a **null**.

La tipologia **Attack** è quella caratterizzata dalla struttura più articolata. Per ciascun messaggio vengono infatti memorizzati il codice identificativo e il nome dell'attacco, il protocollo di rete coinvolto e la durata dell'esecuzione. Lo schema permette inoltre di rappresentare gli argomenti forniti all'attacco attraverso il numero di elementi (**argc**) e i relativi valori (**argv**). Infine, vengono memorizzati il numero dei bersagli coinvolti e il relativo elenco di indirizzi IP di destinazione.

IP Schema

Lo schema include inoltre un array dedicato alle informazioni relative agli indirizzi IP coinvolti negli attacchi, che può rimanere vuoto qualora non siano disponibili ulteriori informazioni sui bersagli. Per ciascun indirizzo è possibile memorizzare dati relativi sia alla sua localizzazione sia alla rete di appartenenza.

In particolare, lo schema consente di rappresentare informazioni geografiche a diversi livelli, dal continente fino alla città, oltre a dati di geolocalizzazione quali latitudine, longitudine e *timezone*. Sono inoltre previste informazioni sulla rete di appartenenza dell'indirizzo IP, con la relativa notazione CIDR e i principali attributi dell'*Autonomous System* (AS), quali numero identificativo (ASN), nome e dominio. Lo schema include infine informazioni relative alla risoluzione DNS e ad altri eventuali metadati associati all'indirizzo.

IP Lookup

L'arricchimento dei dati relativi agli indirizzi IP deve essere gestito dai singoli PYTHON MODULE; tuttavia, per evitare la duplicazione del codice e garantire uniformità nelle modalità di recupero delle informazioni, questa funzionalità è stata centralizzata in un componente dedicato, **IPLookup**, condiviso tra i diversi moduli e richiamabile quando necessario.

¹⁸<https://json-schema.org/draft/2020-12/json-schema-core.html>

Il modulo `IPLookup` implementa una classe progettata per interrogare database locali in formato `.mmdb` (*MaxMind DB*), quali i database *IPinfo Lite*¹⁹ e *GeoLite2*, nelle varianti *City*, *Country* e *ASN*²⁰, così da estrarre le informazioni associate agli indirizzi IP. L'interazione con questi dataset avviene tramite l'apposita libreria Python *maxminddb*²¹, mentre i database sono aggiornati frequentemente e distribuiti gratuitamente, secondo le relative condizioni di licenza^{22 23}.

5.8 Code Templates

Come suggerito durante la fase di progettazione, per separare le responsabilità relative alla cattura e all'elaborazione dei messaggi vengono utilizzati due file distinti, dedicati rispettivamente alla fase di cattura e a quella di parsing dei messaggi secondo il `MESSAGE_SCHEMA`. Il punto di ingresso principale è il `BOTNET_MODULE`, che implementa la `BotNetInterface`.

Per rappresentare a livello di codice la classificazione dei messaggi viene utilizzata un'enumerazione Python²⁴, `MessageType`, che definisce le quattro tipologie di messaggio individuate nello schema.

Infine, poiché il primo modulo utilizza funzionalità di logging, mentre il secondo dipende dalla classe centralizzata `IPLookup`, alla `BotNetInterface` è stato aggiunto un costruttore che permette di fornire tali oggetti durante l'inizializzazione. L'interfaccia completa è riportata nelle appendici.

5.8.1 BotNet Module

Il template completo del modulo è riportato nell'appendice. La responsabilità del `BOTNET_MODULE` consiste nell'implementare i metodi definiti dalla `BotNetInterface`, rispettandone i contratti, e nel fornire lo scheletro del modulo principale che dovrà essere completato in funzione dello specifico protocollo analizzato. Il template definisce pertanto le funzioni del modulo e le modalità con cui le diverse fasi dell'interazione con l'infrastruttura C&C devono essere implementate.

La struttura del modulo può essere descritta secondo le seguenti aree:

Attributi di classe. Contengono i parametri statici specifici del protocollo, quali l'*hostname* o l'indirizzo IP del server C&C, le sequenze utilizzate durante l'handshake, i messaggi o i parametri relativi agli acknowledgment e agli heartbeat e, più in generale, i valori costanti necessari alla comunicazione.

Costruttore `__init__`. Inizializza le risorse e i componenti necessari all'esecuzione del modulo, tra cui il *socket*, il *buffer* utilizzato per la ricezione dei dati, il *logger* e il modulo `MessageParser`, responsabile dell'elaborazione dei messaggi.

Metodo `connect`. Stabilisce la connessione con l'endpoint C&C e avvia, mediante il metodo dedicato *authentication*, la sequenza di inizializzazione prevista dal protocollo. Questa fase permette al modulo di registrarsi presso il server di C&C, simulando il comportamento del dispositivo di una potenziale vittima.

¹⁹<https://ipinfo.io/developers/ipinfo-lite-database>

²⁰<https://dev.maxmind.com/geoip/docs/databases/>

²¹<https://maxminddb.readthedocs.io/en/latest/>

²²<https://ipinfo.io/developers/ipinfo-lite-database> (CC BY-SA 4.0)

²³<https://www.maxmind.com/en/geolite/eula> (EULA con CC BY-SA 4.0)

²⁴<https://docs.python.org/3/library/enum.html>

Metodo sniff. Rimane in ascolto dei dati provenienti dal server C&C e gestisce, quando previsto, l'invio periodico dei messaggi di keepalive tramite il metodo dedicato `heartbeat`. I dati ricevuti vengono eventualmente sottoposti a un pre-processing tramite `preprocess_stream` e sono successivamente aggiunti al buffer. Da quest'ultimo, tramite `extract_packets`, vengono estratti i pacchetti, che possono essere quindi decodificati o decifrati tramite `decode_packet`. Dai pacchetti elaborati vengono estratti i singoli messaggi mediante `extract_messages`, per poi essere classificati secondo il `MessageType` tramite `get_message_type`. Inoltre, quando previsto dal protocollo, `handle_message` permette di eseguire l'azione attesa dal server, in risposta al messaggio ricevuto, come ad esempio l'invio di un acknowledgment. Il parsing del contenuto del messaggio viene invece demandato al `MessageParser` tramite il metodo `parse_message`, che produce un dizionario Python, successivamente restituito tramite `yield`. L'utilizzo di un dizionario garantisce inoltre la compatibilità diretta con la successiva serializzazione in formato JSON, per la quale sarà sufficiente usare la funzione `json.dumps` ²⁵.

Per rendere il template del modulo direttamente utilizzabile nella successiva generazione del codice, tutti i metodi sono accompagnati da *docstring* che ne descrivono il contratto, il comportamento atteso e le operazioni che devono essere implementate. In questo modo, nei prompt forniti al LLM è sufficiente riportare una descrizione più generale del protocollo, senza dover specificare nuovamente tutti i dettagli tecnici. Nel template sono inoltre esplicitamente indicati gli eventi per i quali è previsto il logging, realizzato l'omonima libreria Python ²⁶, così da mantenere coerente anche questo aspetto nella versione finale del modulo.

Una caratteristica particolare del template è l'implementazione iniziale dei metodi ausiliari mediante la keyword `pass`, utilizzata come *placeholder*, oppure restituendo invariati gli oggetti ricevuti. Questa scelta permette di esplicitare al LLM quali funzionalità sono opzionali e dipendenti dallo specifico protocollo: se una determinata funzionalità è prevista dal protocollo, allora il relativo metodo dovrà essere sovrascritto implementandone la logica specifica; in caso contrario, il comportamento predefinito del template può essere mantenuto invariato.

5.8.2 MessageParser Module

Il template completo del modulo è riportato nell'appendice. La responsabilità del `MESSAGEPARSER MODULE` consiste nell'implementare il metodo `parse_message`, richiamato dal `BOTNET MODULE`. Tale metodo esegue il parsing di un messaggio e restituisce un dizionario Python conforme alla struttura definita dal `MESSAGESCHEMA`, pronto per essere sottoposto alla relativa validazione.

Il metodo `parse_message` provvede inizialmente all'estrazione delle informazioni comuni alle diverse tipologie di messaggio, per poi invocare il metodo dedicato all'elaborazione della specifica categoria: `parse_ack`, `parse_command`, `parse_heartbeat` o `parse_attack`. Nel caso dei messaggi di tipo `Attack`, il parser recupera inoltre le informazioni associate agli indirizzi IP dei bersagli dell'attacco mediante la classe `IPLookup`.

Il modulo implementa inoltre una logica di fallback nel caso in cui il parsing specifico non abbia esito positivo. Se uno dei metodi dedicati non riesce a interpretare il messaggio o incorre in un errore,

²⁵<https://docs.python.org/3/library/json.html>

²⁶<https://docs.python.org/3/library/logging.html>

questo viene classificato come un messaggio generico **Command**, garantendo comunque la produzione di un risultato conforme allo schema previsto.

Anche per il MESSAGEPARSER MODULE valgono gli stessi principi già adottati per il BOTNET MODULE in merito alle descrizioni tramite docstring e alle modalità di implementazione.

5.9 Prompt Archive

- Si presentano tutti i prompt, cominciando per tutti dal system prompt e poi proseguendo con gli user prompt.
- I prompt verranno riportati tra le appendici, completi oppure con alcuni snippet.
- per l'introduzione dei materiali viene fatta la context injection, prevedendo dei "tag" che vengono valorizzati dinamicamente in base all'uso.

5.9.1 Renaming

System Prompt.

Start Prompt.

Continue Prompt.

5.9.2 Reversing

System Prompt.

1st half Prompt.

2nd half Prompt.

5.9.3 Code Agent

System Prompt. Definisce come ruolo del LLM quello di agente specializzato nella generazione di codice Python per moduli di sicurezza di rete. Richiede la produzione di codice completo ed eseguibile, conforme alle specifiche, alle informazioni e ai template forniti. Il prompt descrive inoltre le modalità di estrazione delle informazioni a partire dalla documentazione fornita e stabilisce gli standard di implementazione e qualità del codice da seguire. In particolare, richiede l'utilizzo di Python 3.11+, l'adozione di buone pratiche di sviluppo, con modularità, separazione delle responsabilità tra i componenti e chiarezza del codice, oltre a una gestione adeguata delle eccezioni e del logging.

BotNet Prompt. Definisce il compito di generazione del modulo di cattura, specificando che l'implementazione deve essere contenuta in un singolo file e ricostruire i layer di comunicazione e trasporto della botnet. Il prompt fornisce quindi i materiali di riferimento, ciascuno affiancato da una breve descrizione: la **BotNetInterface**, i **SRE Reports**, il template del **BotNet Module** e l'enumerazione **MessageType**. Vengono successivamente definiti i requisiti per l'implementazione

dei metodi `connect` e `sniff`, specificando in generale il comportamento atteso. Infine, si richiede di rispettare il template esistente e di implementare esclusivamente le funzionalità previste dalla specifica, escludendo qualsiasi comportamento offensivo e restituendo in output il file completo.

MessageParser Prompt. Poiché questa generazione segue quella del `BotNet Module`, il prompt si limita a estendere i materiali di riferimento con il `MessageSchema` e il template del `MessageParser`. Il compito definito è la generazione del modulo di parsing, il cui punto di ingresso è rappresentato dal metodo `parse_message`, responsabile di indirizzare i messaggi verso i parser specifici. Viene inoltre richiesto di utilizzare il tipo di messaggio generico `Command` come *fallback* in caso di errore durante il parsing. Anche in questo caso, il prompt richiede il rispetto del template, delle richieste definite e del formato di output previsto.

6

Validazione e test

6.1 Obiettivi della validazione

6.1.1 Strategia di validazione

Le prestazioni non sono oggetto della valutazione in quanto.

6.2 Ambiente sperimentale

6.2.1 LLM

Qwen 3.6 27B • Architettura Dense e parametri

- Contesto
- Hardware (1*A100)
- vLLM Recipe (quantizzazione)
- Usage Parameters (0.8)

DeepSeek-V4-Flash-Preview • Architettura MoE e parametri

- Contesto
- Hardware (4*H100)
- vLLM Recipe (quantizzazione)
- Usage Parameters (1.0 thinking, 0.8)

6.3 Definizione delle metriche di valutazione

6.3.1 Complessità del campione

6.3.2 Efficacia dell'attività di rinomina

6.3.3 Accuratezza del recupero degli artefatti

6.3.4 Qualità del codice generato

6.4 Campagna sperimentale

6.4.1 Dataset

Famiglia Mirai

Snippet MD K.B.

6.4.2 Descrizione dei campioni

Hash, architettura, dimensione, famiglia

Difficoltà

6.5 Risultati sperimentali

6.5.1 Statistiche di esecuzione

Token

6.5.2 Metriche

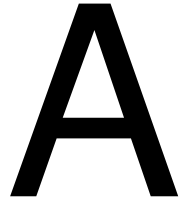
Tabelle

Grafici

7

Conclusioni

7.1 Sviluppi Futuri



Titolo della prima appendice

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Bibliografia

- [1] Michael Sikorski e Andrew Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, San Francisco, CA, febbraio 2012.
- [2] Sérgio S. C. Silva, Rodrigo M. P. Silva, Raquel C. G. Pinto, e Ronaldo M. Salles. Botnets: A survey. *Computer Networks*, 57(2):378–403, febbraio 2013.
- [3] Manos Antonakakis, Tim April, Michael Bailey, Matthew Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, Deepak Kumar, Chaz Lever, Zane Ma, Joshua Mason, Damian Menscher, Chad Seaman, Nick Sullivan, Kurt Thomas, e Yi Zhou. Understanding the Mirai botnet. In *Proceedings of the 26th USENIX Security Symposium (USENIX Security 2017)*, pp. 1093–1110, Vancouver, BC, Canada, agosto 2017. USENIX Association.
- [4] Kara Nance e Chris Eagle. *The Ghidra Book: The Definitive Guide*. No Starch Press, San Francisco, CA, 2 edizione, marzo 2026.
- [5] Andreas Moser, Christopher Kruegel, e Engin Kirda. Limits of static analysis for malware detection. In *Proceedings of the 23rd Annual Computer Security Applications Conference (ACSAC 2007)*, pp. 421–430, Miami Beach, FL, USA, dicembre 2007. IEEE Computer Society.
- [6] Manuel Egele, Theodoor Scholte, Engin Kirda, e Christopher Kruegel. A survey on automated dynamic malware-analysis techniques and tools. *ACM Computing Surveys*, 44(2):1–42, febbraio 2012.
- [7] Numa Dhamani e Maggie Engler. *Introduction to Generative AI*. Manning Publications, Shelter Island, NY, 2 edizione, gennaio 2026.
- [8] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, e Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [9] Model Context Protocol. What is the model context protocol (mcp)?, 2024. Official documentation.
- [10] Hanzhuo Tan, Qi Luo, Jing Li, e Yuqun Zhang. LLM4Decompile: Decompiling binary code with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 3473–3487, Miami, Florida, USA, novembre 2024. Association for Computational Linguistics.
- [11] Guoqiang Chen, Huiqi Sun, Daguang Liu, Zhiqi Wang, Qiang Wang, Bin Yin, Lu Liu, e Lingyun Ying. ReCopilot: Reverse engineering copilot in binary analysis, maggio 2025.

- [12] Brian Caswell, Dustin Frazee, Sarah Smith, Rodrigo Racanicki, Tim Middleton-Sally, Shelby Hayes, Stanley He, Katy Smith, Bhakta Pradhan, e Mike Walker. Project ire autonomously identifies malware at scale, agosto 2025.
- [13] Xingyuan Wei, Ziyang Yu, Lihao Liu, Degang Sun, Wentao Li, e Yan Wang. Masdroid: A multi-agent system for enhancing the analysis of android malware. In *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 14147–14151, 2026.
- [14] Zion Leonahenahe Basque, Samuele Doria, Ananta Soneji, Wil Gibbs, Adam Doupé, Yan Shoshitaishvili, Eleonora Losiouk, Ruoyu Wang, e Simone Aonzo. Decompiling the synergy: An empirical study of human–LLM teaming in software reverse engineering. In *Proceedings of the 33rd Network and Distributed System Security Symposium (NDSS 2026)*, San Diego, CA, USA, febbraio 2026. The Internet Society. Distinguished Paper Award.
- [15] T. J. O’Connor. *Violent Python: A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers*. Syngress, Waltham, MA, USA, 2012.
- [16] Luciano Ramalho. *Fluent Python: Clear, Concise, and Effective Programming*. O’Reilly Media, 2 edizione, 2022.
- [17] Object Management Group. OMG Unified Modeling Language (OMG UML) Version 2.5.1. Relazione Tecnica formal/2017-12-05, Object Management Group, dicembre 2017.
- [18] Docker Inc. *What is a container?* Docker Inc., 2026. Docker Docs: Get started, Docker concepts.
- [19] Austin Wright, Henry Andrews, Ben Hutton, e Greg Dennis. Json schema: A media type for describing json documents. Relazione tecnica, JSON Schema, 2020. Overview and core documentation for Draft 2020-12.